

SOFTWARE REQUIREMENTS SPECIFICATION

SRS

CES Service Management System

Layihə Adı	CES Service Management System
Sənəd Növü	Software Requirements Specification (SRS)
Versiya	1.0 — Part 1: Foundation
Tarix	29 İyun 2026
Hazırlayan	Sarkhan Babayev
BRD İstinadu	CES Service BRD v1.0
Status	Draft

Part 1 — Foundation

System Overview • Technology Stack • Database Conventions • Auth Architecture • Multi-Branch Model

1. System Overview

1.1 Məqsəd

Bu sənəd CES Service Management System platformasının tam texniki tələblərini əhatə edən Software Requirements Specification (SRS)-dir. Sənəd BRD v1.0 əsasında hazırlanmış və development komandası üçün birincil texniki istinad rolunu oynayır.

SRS aşağıdakıları əhatə edir: API endpoint spesifikasiyası (HTTP metod, URL, request/response sxemi), verilənlər bazı cədvəlləri (DDL), entity əlaqə diaqramları, biznes məntiq qaydaları, doğrulama (validation) qaydaları, xəta kodları və səlahiyyət modeli.

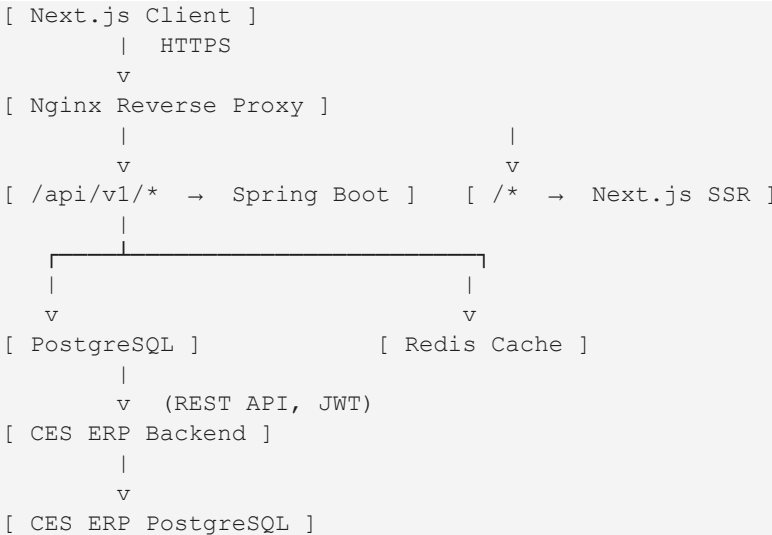
1.2 Sistem Arxitektura Xülasəsi

İki müstəqil platforma: CES ERP (icarə) və CES Service (texniki servis). İntegrasiya API üzərindən — shared DB yoxdur.

Qat (Layer)	CES ERP	CES Service
Frontend	React (mövcud)	Next.js 14+ (App Router)
Backend	Spring Boot / Java (mövcud)	Spring Boot 3.x / Java 21
Verilənlər bazı	PostgreSQL (mövcud)	PostgreSQL 16 (ayrıca schema)
Cache	Redis (mövcud)	Redis 7 (shared instance)
Migration	Flyway (mövcud)	Flyway 10
Auth	JWT (mövcud)	JWT — müstəqil token
Reverse Proxy	Nginx (mövcud)	Nginx — eyni instance, ayrı location block
Container	Docker / Compose (mövcud)	Docker / Compose — ayrı servis
İntegrasiya	—	CES ERP REST API (JWT)

1.3 Yüksək Səviyyəli Arxitektura Axını

Aşağıdakı axın CES Service sisteminin əsas komponentlərinin bir-biri ilə əlaqəsini göstərir:



1.4 Əhatə Dairəsi — 19 Modul

Part	Modul	Ad	Əhatə
1	M01	Login & Naviqasiya	JWT auth, sistem seçimi, session
1	M02	Dashboard	KPI, bildirişlər, real-vaxt panel
2	M03	Qaraj Modulu	Şirkət + investor + müştəri texnikaları
2	M04	Müştəri İdarəetməsi	CRUD, CES ERP sinxronizasiya
2	M05	Servis Sorğuları	3 növ sorğu: Pullu / Şirkət / Vasitəçilik
2	M06	Work Order	Status axını, tapşırıq, bağlama
2	M07	Xərc & Marja	Detal xərci, iş haqqı, marja hesabı
2	M08	Motosaat	Manual/auto qeydiyyat, validasiya
2	M18	Texniki Baxış	Checklist, trigger, avtomatik WO
2	M19	Anbar & Ehtiyat Hissələri	Layer qovluq, stok, WO-bağlı çıxış
3	M09	Arxiv	Immutable qeydlər, axtarış
3	M10	Sənəd Generasiyası	PDF: Servis Aktı, Faktura, Təklif
3	M11	Hesabatlar	Gəlir, xərc, mənfəət, export
3	M12	Bildiriş Mərkəzi	In-app bildirişlər, xəbərdarlıqlar
3	M13	CES ERP İnteqrasiyası	Müştəri API, Mühəsibatlıq push
3	M14	Mobil Tətbiq (Faza 1)	Motosaat — REST API
4	M15	İstifadəçi İdarəetməsi	CRUD, status, branch, rol
4	M16	Rol & İcazə	Dinamik RBAC, Permission Matrix
4	M17	Audit Log	Immutable log, bütün hadisələr

2. Technology Stack

2.1 Backend — Spring Boot 3.x / Java 21

Komponent	Versiya / Artefakt	İstifadə
Java	21 (LTS)	Runtime — Virtual Threads dəstəyi
Spring Boot	3.3.x	Əsas framework
Spring Security	6.x	JWT filter zənciri, RBAC authorization
Spring Data JPA	3.x	ORM — PostgreSQL
Spring Data Redis	3.x	Cache, session, rate-limit
Flyway	10.x	DB migration — versiyalı SQL skriptlər
MapStruct	1.5.x	Entity ↔ DTO çevirmə
Lombok	latest	Boilerplate azaltma
jjwt (JJWT)	0.12.x	JWT imzalama / doğrulama
iText / JasperReports	latest	PDF sənəd generasiyası (M10)
SpringDoc OpenAPI	2.x	Swagger UI — API sənədləşməsi
Testcontainers	1.19.x	Integration test — Docker-daxili PostgreSQL

2.2 Frontend — Next.js 14+

Komponent	Versiya	İstifadə
Next.js	14+ (App Router)	SSR / SSG / RSC — əsas framework
React	18+	UI komponent kitabxanası
TypeScript	5.x	Statik tipləmə
Tailwind CSS	3.x	Utility-first CSS — CES ERP ilə eyni design system
Shadcn/ui	latest	Komponent kitabxanası — Tailwind əsaslı
TanStack Query	5.x	Server state idarəetməsi, cache, invalidation
Zustand	4.x	Client state (auth, branch seçimi)
React Hook Form + Zod	latest	Form idarəetməsi + schema validation
Recharts	2.x	Dashboard chart-lar
next-auth / jose	latest	JWT session idarəetməsi — Next.js tərəfi
Axios	1.x	HTTP client — API sorğuları

2.3 Verilənlər Bazı — PostgreSQL 16

Komponent	Versiya	İstifadə
PostgreSQL	16	Əsas relational DB
Schema adı	ces_service	CES ERP-dən ayrı schema

Komponent	Versiya	İstifadə
UUID extension	pgcrypto / gen_random_uuid()	PK generasiyası
JSONB	native	Dinamik metadata sahələri üçün
pg_trgm	extension	Full-text axtarış (trigram index)
Row-Level Security	optional	Gələcək multi-tenant gücləndirilməsi üçün

2.4 Cache — Redis 7

- JWT blacklist (logout / token revoke)
- Session cache — istifadəçi profili, rol + icazə siyahısı
- Rate limiting (brute-force qorunması)
- Dashboard KPI cache — 60 saniyəlik TTL
- Motosaat son dəyər cache — tez-tez oxunan sahə
- CES ERP müştəri bazası cache — 5 dəqiqəlik TTL

Redis cluster rejimi bu mərhələdə tələb olunmur. Tək Redis instance + AOF persistence kifayətdir.

2.5 İnfrastruktur

Komponent	Versiya	Konfigurasiya
Docker	26+	Hər servis ayrı container: backend, frontend, postgres, redis
Docker Compose	2.x	Local dev + staging. Production üçün ayrı compose fayl
Nginx	1.26+	Reverse proxy: /api/v1/* → backend:8080, /* → frontend:3000
Nginx — SSL	Let's Encrypt	TLS termination Nginx-də — backend plain HTTP
Flyway	10.x	Startup-da avtomatik migration — Spring Boot ilə integrasiya

2.6 Nginx Konfigurasiya Nümunəsi

```
server {
    listen 443 ssl;
    server_name service.ces.az;

    # API → Spring Boot
    location /api/ {
        proxy_pass          http://backend:8080;
        proxy_set_header    Authorization $http_authorization;
        proxy_set_header    X-Branch-Id   $http_x_branch_id;
    }

    # Everything else → Next.js
    location / {
        proxy_pass          http://frontend:3000;
    }
}
```

2.7 Flyway Migration Konvensiya

Bütün DB dəyişiklikləri Flyway migration skriptləri ilə idarə olunur. Heç bir manual DDL qadağandır.

Fayl adı nümunəsi	Növ	Qayda
V1__init_schema.sql	Versioned	Bir dəfə — geri çevrilmir
V2__add_branch_table.sql	Versioned	Ardıcıl nömrələmə, boşluq olmaz
R__refresh_views.sql	Repeatable	Hər dəfə checksum dəyişsə icra olunur
U2__rollback_branch.sql	Undo (optional)	Yalnız Flyway Teams — bu mərhələdə deyil

Qayda: Mövcud migration faylını heç vaxt dəyişmə. Yeni dəyişiklik = yeni versioned skript.

3. Verilənlər Bazı Konvensiyaları

3.1 Ümumi Qaydalar

Qayda	Detal
Schema adı	ces_service — CES ERP schemadan tam ayrı
Cədvəl adları	snake_case, cəm (plural): work_orders, spare_parts
Sütun adları	snake_case: created_at, branch_id, unit_cost
Primary Key	UUID — gen_random_uuid() default
Foreign Key	UUID tipli, ON DELETE davranışı hər cədvəldə açıq yazılır
Timestamps	created_at, updated_at — hər cədvəldə məcburi, DEFAULT NOW()
Soft delete	deleted_at TIMESTAMPTZ NULL — fiziki silmə yoxdur
Audit fields	created_by UUID, updated_by UUID — users.id FK
Boolean	is_ prefiksi: is_active, is_deleted
Enum	PostgreSQL ENUM tipi — dəyişikliyə açıq olsun deyə VARCHAR(50) + CHECK
İndeks	FK sütunları, axtarış sütunları (branch_id, status, created_at)
Money	NUMERIC(15,2) — FLOAT qadağandır (yuvarlama xətası)
Percent	NUMERIC(5,2) — 0.00 – 100.00 aralığı

3.2 Universal Baza Cədvəl Şablonu

Sistemdəki hər cədvəl aşağıdakı sütunları məcburi olaraq ehtiva edir:

```
CREATE TABLE ces_service.<table_name> (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id   UUID          NOT NULL REFERENCES branches(id),

  -- ... cədvələ xüsusi sütunlar ...

  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  deleted_at  TIMESTAMPTZ   NULL,
  created_by  UUID          NOT NULL REFERENCES users(id),
  updated_by  UUID          NOT NULL REFERENCES users(id)
);

-- Soft delete üçün partial index
CREATE INDEX idx_<table>_active ON <table>(branch_id)
WHERE deleted_at IS NULL;
```

Soft delete: deleted_at IS NOT NULL olan qeydlər normal sorğularda görünür. Spring Data JPA-da @Where(clause = "deleted_at IS NULL") annotasiyası tətbiq olunur.

3.3 UUID İstifadə Sxemi

- Bütün primary key-lər UUID — PostgreSQL-in gen_random_uuid() funksiyası

- Java tərəfindən UUID yaratmaq da mümkündür (UUID.randomUUID()) — hər iki yanaşma qəbul edilir
- CES ERP-dən gələn xarici entity ID-ləri (məs. erp_customer_id) ayrı sütunda saxlanılır — CES Service UUID-si ilə qarışdırılmır

Nümunə: Müştəri cədvəlinin xarici ID idarəsi

```
CREATE TABLE ces_service.customers (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    erp_customer_id   UUID NULL,      -- CES ERP-dəki UUID (sinxronizasiya üçün)
    branch_id         UUID NOT NULL REFERENCES branches(id),
    full_name         VARCHAR(255) NOT NULL,
    voen              VARCHAR(20) NULL,
    phone             VARCHAR(50) NULL,
    email             VARCHAR(255) NULL,
    -- ... digər sahələr
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    deleted_at        TIMESTAMPTZ NULL,
    created_by        UUID NOT NULL REFERENCES users(id),
    updated_by        UUID NOT NULL REFERENCES users(id)
);
CREATE UNIQUE INDEX idx_customers_erp_id
    ON customers(erp_customer_id) WHERE erp_customer_id IS NOT NULL;
```

3.4 Enum Konvensiyası

PostgreSQL native ENUM əvəzinə VARCHAR(50) + CHECK constraint — belə ki gələcəkdə yeni dəyər əlavə etmək migration ilə asandır:

```
-- Təvsiyə olunan yanaşma:
status VARCHAR(50) NOT NULL
CHECK (status IN
('NEW', 'ASSIGNED', 'IN_PROGRESS', 'ON_HOLD', 'COMPLETED', 'CLOSED', 'CANCELLED')),

-- Java tərəfindən:
@Enumerated(EnumType.STRING)
@Column(name = "status", nullable = false)
private WorkOrderStatus status;
```

3.5 NUMERIC Konvensiyası (Pul və Faiz)

Tip	PostgreSQL	İstifadə halı
Pul məbləği	NUMERIC(15,2)	unit_cost, sell_price, total_amount
Faiz / Marja	NUMERIC(5,2)	margin_percent: 0.00 – 100.00
Miqdar	NUMERIC(10,3)	quantity: litr, kq kimi kəsr ola bilər
Motosaat	NUMERIC(10,1)	engine_hours: 0.0 – 99999.9
Java mapping	BigDecimal	Double / Float heç vaxt işlədilmir

4. Autentifikasiya & Avtorizasiya Arxitekturası

4.1 JWT Token Sxemi

CES Service müstəqil JWT token istifadə edir. CES ERP tokeni qəbul edilmir — istifadəçi hər iki sistemdə ayrıca giriş edir.

Access Token — Payload strukturu:

```
{
  "sub": "550e8400-e29b-41d4-a716-446655440000", // user UUID
  "email": "user@ces.az",
  "branch_id": "7c9e6679-7425-40de-944b-e07fc1f90ae7", // aktiv branch
  "roles": ["SERVICE_MANAGER"],
  "permissions": ["WO_CREATE", "WO_READ", "WO_UPDATE", "CHANGE_MARGIN"],
  "iat": 1719650000,
  "exp": 1719653600, // 1 saat
  "jti": "unique-token-id"
}
```

Refresh Token — Payload strukturu:

```
{
  "sub": "550e8400-e29b-41d4-a716-446655440000",
  "jti": "unique-refresh-id",
  "iat": 1719650000,
  "exp": 1720254800 // 7 gün
}
```

Parametr	Dəyər	Qeyd
Access token TTL	1 saat	Qısa ömür — security üçün
Refresh token TTL	7 gün	Redis-də saxlanılır
İmzalama alqoritmi	HS256 (başlanğıc) / RS256 (tövsiyə)	RS256 açıq açar paylaşımı üçün daha təhlükəsiz
Token yeri	Authorization: Bearer <token>	Cookie-də saxlamaq opsional (Next.js HttpOnly cookie)
Blacklist	Redis — jti ilə	Logout / şifrə dəyişikliyi / deaktiv
Branch dəyişikliyi	Yeni token alınır	Refresh endpoint-indən branch_id ilə

4.2 Auth API Endpoint-ləri

Metod	URL	Açıqlama	Auth
POST	/api/v1/auth/login	Giriş — access + refresh token qaytarır	Yoxdur
POST	/api/v1/auth/refresh	Refresh token ilə yeni access token	Refresh token
POST	/api/v1/auth/logout	Token blacklist-ə əlavə edilir	Bearer
POST	/api/v1/auth/switch-branch	Fərqli branch üçün yeni token	Bearer

Metod	URL	Açıqlama	Auth
POST	/api/v1/auth/forgot-password	Şifrə sıfırlama e-mail göndərir	Yoxdur
POST	/api/v1/auth/reset-password	Yeni şifrə təyin edir	Reset token

POST /api/v1/auth/login — Request:

```
{
  "email": "user@ces.az",
  "password": "SecureP@ss123"
}
```

POST /api/v1/auth/login — Response 200:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiJ9...",
  "refresh_token": "eyJhbGciOiJIUzI1NiJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "user": {
    "id": "550e8400-...",
    "full_name": "Əli Hüseynov",
    "email": "user@ces.az",
    "branch_id": "7c9e6679-...",
    "branches": [{ "id": "...", "name": "Bakı filialı" }],
    "roles": ["SERVICE_MANAGER"],
    "permissions": ["WO_CREATE", "WO_READ", "WO_UPDATE", "CHANGE_MARGIN"]
  }
}
```

4.3 Spring Security Filter Zənciri

Hər API sorğusu aşağıdakı filtr ardıcılığından keçir:

```
Request
├─ CorsFilter (OPTIONS preflight-ı buraxır)
├─ RateLimitFilter (Redis – IP başına limit)
├─ JwtAuthenticationFilter (token parse + validate)
│   ├── jti blacklist yoxla (Redis)
│   ├── exp yoxla
│   └─ SecurityContext-ə set et
├─ BranchContextFilter (X-Branch-Id header → ThreadLocal)
├─ Controller / Service
│   └─ @PreAuthorize("hasPermission(...)") yoxlaması
```

X-Branch-Id header: Hər API sorğusunda client aktiv branch UUID-sini göndərir. Filter bunu ThreadLocal-a yazır — service katmanı bu dəyərdən istifadə edir.

4.4 Avtorizasiya Yoxlaması — @PreAuthorize

Controller metodları Spring Security @PreAuthorize annotasiyası ilə qorunur. İcazə yoxlaması permission kodu əsasında aparılır:

```
// Permission yoxlaması:
@GetMapping("/{id}")
@PreAuthorize("hasAuthority('WO_READ')")
public ResponseEntity<WorkOrderResponse> getById(@PathVariable UUID id) { ... }

// Xüsusi biznes icazəsi:
```

```
@PatchMapping("/{id}/margin")
@PreAuthorize("hasAuthority('CHANGE_MARGIN')")
public ResponseEntity<Void> updateMargin(...) { ... }

// Mürəkkəb şərt:
@DeleteMapping("/{id}")
@PreAuthorize("hasAuthority('WO_DELETE') and @branchGuard.check(#id)")
public ResponseEntity<Void> delete(@PathVariable UUID id) { ... }
```

@branchGuard.check(#id): Entity-nin branch_id-si ilə cari istifadəçinin branch_id-sinin eyni olduğunu yoxlayan custom SpEL bean.

4.5 Brute-Force Qorunması

- Redis-də IP bazlı sayıcı: hər uğursuz giriş +1
- 5 uğursuz cəhd → 15 dəqiqəlik blok
- 30 uğursuz cəhd (fərqli IP-lərdən) → account lock → Admin müdaxiləsi tələb olunur
- Uğurlu giriş → sayacı sıfırla
- Bütün blok hadisələri Audit Log-da qeydə alınır

5. Multi-Branch (Çoxşöbəli) Arxitektura

5.1 Konsepsiya

CES Service çoxşöbəli (multi-branch) quruluşda işləyir. Hər şöbə (branch) tam müstəqil data dairəsinə malikdir — bir şöbənin istifadəçisi digər şöbənin məlumatlarını görə bilmir. Bu, tək database / tək schema daxilində row-level tenant ayrımı ilə həyata keçirilir.

Yanaşma: Shared Database + Shared Schema + branch_id sütunu (Row-Level Tenancy). Hər cədvəldə branch_id məcburi sahədir.

5.2 Branches Cədvəli

```
CREATE TABLE ces_service.branches (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(255) NOT NULL,
  code        VARCHAR(50)   NOT NULL UNIQUE, -- məs. "BAK", "GNJ"
  address     TEXT          NULL,
  phone       VARCHAR(50)   NULL,
  is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  deleted_at  TIMESTAMPTZ   NULL,
  created_by  UUID          NOT NULL REFERENCES users(id),
  updated_by  UUID          NOT NULL REFERENCES users(id)
);
```

5.3 İstifadəçi ↔ Branch Əlaqəsi

İstifadəçi birdən çox şöbəyə aid ola bilər. Aktiv şöbə seçimi token daxilindədir.

```
CREATE TABLE ces_service.user_branches (
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  branch_id   UUID NOT NULL REFERENCES branches(id) ON DELETE CASCADE,
  is_default  BOOLEAN NOT NULL DEFAULT FALSE,
  PRIMARY KEY (user_id, branch_id)
);
```

İstifadəçi sistemə daxil olduqda default branch seçilir. Başqa branch-a keçmək üçün /api/v1/auth/switch-branch endpoint-i çağrılır — yeni token verilir.

5.4 Service Katmanında Branch Filtri

Spring Data JPA repository-lərində bütün sorğular branch_id ilə filtrlənir. Bu, @Query annotasiyası və ya Specification pattern ilə həyata keçirilir:

```
// Yanaşma 1: Repository metodunda explicit branch_id
List<WorkOrder> findByBranchIdAndDeletedAtIsNull(UUID branchId);

// Yanaşma 2: BranchContext bean ilə avtomatik
@Component
public class BranchContext {
  private static final ThreadLocal<UUID> current = new ThreadLocal<>();
  public static void set(UUID id) { current.set(id); }
  public static UUID get()       { return current.get(); }
  public static void clear()      { current.remove(); }
}

// BaseEntity-də @Filter ilə (Hibernate):
```

```
@FilterDef(name="branchFilter",
    parameters=@ParamDef(name="branchId", type=UUID.class))
@Filter(name="branchFilter", condition="branch_id = :branchId")
@MappedSuperclass
public abstract class BaseEntity { ... }
```

Təvsiyə: Yanaşma 1 (explicit branch_id) daha şəffaf və test edilməsi asandır. Yanaşma 2 daha az boilerplate, lakin Hibernate Filter-in yan təsirlərinə diqqət lazımdır.

5.5 Cross-Branch İcazələri

Bəzi rollar birdən çox şöbənin məlumatlarını görə bilər. Bu, xüsusi permission ilə idarə olunur:

İcazə kodu	Mənası	Nümunə istifadəçi
BRANCH_VIEW_OWN	Yalnız öz şöbəsini görür (default)	Bütün istifadəçilər
BRANCH_VIEW_ALL	Bütün şöbələrin məlumatlarını görür	CEO, Baş Mühasib
BRANCH_MANAGE	Şöbə yarada / dəyişdirə bilər	System Admin

5.6 API-də Branch Göstəricisi

Client hər API sorğusunda aktiv branch-ı X-Branch-Id header-i ilə göndərir. Backend bu header-i token-dakı branch_id ilə tutuşdurur:

```
GET /api/v1/work-orders HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
X-Branch-Id: 7c9e6679-7425-40de-944b-e07fc1f90ae7
```

```
// BranchContextFilter.java
String branchHeader = request.getHeader("X-Branch-Id");
UUID branchId = UUID.fromString(branchHeader);
// JWT-dəki branch list-ə daxildir yə yox yoxla:
if (!jwtBranches.contains(branchId)) {
    throw new ForbiddenException("Branch access denied");
}
BranchContext.set(branchId);
```

5.7 DB İndeksləri — Multi-Branch

Performans üçün branch_id sütununu ehtiva edən composite index-lər:

```
-- Work Orders: branch + status + created_at (ən çox istifadə olunan filtr)
CREATE INDEX idx_work_orders_branch_status
    ON work_orders(branch_id, status, created_at DESC)
    WHERE deleted_at IS NULL;

-- Vehicles: branch + vehicle_type
CREATE INDEX idx_vehicles_branch_type
    ON vehicles(branch_id, vehicle_type)
    WHERE deleted_at IS NULL;

-- Spare parts: branch + category_node
CREATE INDEX idx_spare_parts_branch_cat
    ON spare_parts(branch_id, category_node_id)
    WHERE deleted_at IS NULL;
```


6. API Konvensiyaları

6.1 URL Strukturu

```
/api/v1/{resource}
/api/v1/{resource}/{id}
/api/v1/{resource}/{id}/{sub-resource}
/api/v1/{resource}/{id}/{sub-resource}/{sub-id}
```

Metod	URL nümunəsi	Mənası
GET	/api/v1/work-orders	Siyahı (paginasiya ilə)
GET	/api/v1/work-orders/{id}	Tək qeyd
POST	/api/v1/work-orders	Yeni yaratma
PUT	/api/v1/work-orders/{id}	Tam yeniləmə
PATCH	/api/v1/work-orders/{id}/status	Qismən yeniləmə (xüsusi əməliyyat)
DELETE	/api/v1/work-orders/{id}	Soft delete
GET	/api/v1/work-orders/{id}/items	Alt-resurs siyahısı
POST	/api/v1/work-orders/{id}/items	Alt-resurs əlavəsi

6.2 Standart Response Formatı

Uğurlu cavab (200 / 201):

```
{
  "success": true,
  "data": { ... },           // tək obyekt və ya siyahı
  "meta": {                  // yalnız siyahılarda
    "page": 1,
    "size": 20,
    "total_pages": 5,
    "total_items": 97
  },
  "timestamp": "2026-06-29T12:00:00Z"
}
```

Xəta cavabı (4xx / 5xx):

```
{
  "success": false,
  "error": {
    "code": "WO_NOT_FOUND",
    "message": "Work order tapılmadı",
    "details": [ ... ]      // validation xətaləri üçün array
  },
  "timestamp": "2026-06-29T12:00:00Z"
}
```

6.3 Paginasiya

Bütün siyahı endpoint-ləri eyni paginasiya parametrlərini qəbul edir:

```
GET /api/v1/work-orders?page=1&size=20&sort=created_at&dir=desc
```

Parametr	Tip	Default	Açıqlama
page	integer	1	Səhifə nömrəsi (1-dən başlayır)
size	integer	20	Səhifə ölçüsü (max: 100)
sort	string	created_at	Sıralama sahəsi
dir	string	desc	asc desc
search	string	null	Açar söz axtarışı (pg_trgm)

6.4 HTTP Status Kodları

Kod	Ad	İstifadə halı
200	OK	GET, PUT, PATCH — uğurlu
201	Created	POST — uğurlu yaratma
204	No Content	DELETE — uğurlu silmə
400	Bad Request	Validation xətası, yanlış format
401	Unauthorized	Token yoxdur, etibarsızdır, müddəti bitib
403	Forbidden	Token var, lakin icazə yoxdur
404	Not Found	Resurs tapılmadı
409	Conflict	Dublikat qeyd (məs. eyni VÖEN)
422	Unprocessable Entity	Biznes qaydası pozulması
429	Too Many Requests	Rate limit — brute-force qorunması
500	Internal Server Error	Gözlənilməz server xətası

6.5 Standart Xəta Kodları

Xəta kodu	HTTP	Mənası
AUTH_INVALID_CREDENTIALS	401	Email / şifrə yanlışdır
AUTH_TOKEN_EXPIRED	401	Access token müddəti bitib
AUTH_TOKEN_REVOKED	401	Token blacklist-ə əlavə edilib
AUTH_ACCOUNT_LOCKED	401	Çox uğursuz cəhd — hesab kilidli
BRANCH_ACCESS_DENIED	403	X-Branch-Id token-dakı branch-lardan deyil
PERMISSION_DENIED	403	Tələb olunan icazə yoxdur
RESOURCE_NOT_FOUND	404	Ümumi — entity tapılmadı
WO_NOT_FOUND	404	Work Order tapılmadı
VEHICLE_NOT_FOUND	404	Texnika tapılmadı

Xəta kodu	HTTP	Mənası
DUPLICATE_VOEN	409	Bu VÖEN artıq mövcuddur
WO_INVALID_STATUS_TRANSITION	422	İcazəsiz status keçidi
STOCK_INSUFFICIENT	422	Stok kifayət etmir
ENGINE_HOURS DECREASING	422	Motosaat azalan dəyər
COST_BELOW_UNIT_COST	422	Satış qiyməti maya dəyərindən aşağı
WO_CLOSE_REQUIRES_COST	422	WO bağlanmaq üçün xərc tələb olunur
RATE_LIMIT_EXCEEDED	429	Çox sorğu — gözlə

7. Spring Boot Layihə Strukturu

7.1 Paket Strukturu

```

com.ces.service
├── CesServiceApplication.java
├── config/
│   ├── SecurityConfig.java
│   ├── RedisConfig.java
│   ├── JpaConfig.java
│   └── OpenApiConfig.java
├── common/
│   ├── entity/
│   │   └── BaseEntity.java           // id, branch_id, timestamps, created_by
│   ├── dto/
│   │   ├── ApiResponse.java
│   │   └── PageResponse.java
│   ├── exception/
│   │   ├── GlobalExceptionHandler.java
│   │   ├── BusinessException.java
│   │   └── ErrorCode.java
│   └── filter/
│       ├── JwtAuthenticationFilter.java
│       ├── BranchContextFilter.java
│       └── RateLimitFilter.java
├── module/
│   ├── auth/
│   │   ├── controller/ AuthController.java
│   │   ├── service/    AuthService.java, JwtService.java
│   │   └── dto/         LoginRequest.java, TokenResponse.java
│   ├── branch/
│   │   ├── entity/     Branch.java
│   │   ├── repository/ BranchRepository.java
│   │   ├── service/    BranchService.java
│   │   ├── controller/ BranchController.java
│   │   └── dto/         BranchRequest.java, BranchResponse.java
│   ├── vehicle/        // M03 Qaraj
│   ├── customer/       // M04 Müştəri
│   ├── service-request/ // M05 Servis Sorğuları
│   ├── work-order/     // M06 Work Order
│   ├── cost/           // M07 Xərc & Marja
│   ├── engine-hours/   // M08 Motosaat
│   ├── inspection/     // M18 Texniki Baxış
│   ├── warehouse/      // M19 Anbar
│   ├── archive/        // M09 Arxiv
│   ├── document/       // M10 Sənəd
│   ├── report/         // M11 Hesabatlar
│   ├── notification/   // M12 Bildiriş
│   ├── erp-integration/ // M13 CES ERP
│   ├── user/           // M15 İstifadəçi
│   ├── role/           // M16 RBAC
│   └── audit/          // M17 Audit Log
└── infrastructure/
    ├── redis/          RedisTokenStore.java, CacheService.java
    ├── erp/            ErpApiClient.java
    └── pdf/            PdfGeneratorService.java
  
```

7.2 BaseEntity

```

@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
@FilterDef(name="branchFilter", ...),
@Filter(name="branchFilter", condition="branch_id = :branchId")
public abstract class BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(name="branch_id", nullable=false, updatable=false)
    private UUID branchId;

    @CreatedDate
    @Column(name="created_at", updatable=false)
    private Instant createdAt;

    @LastModifiedDate
    @Column(name="updated_at")
    private Instant updatedAt;

    @Column(name="deleted_at")
    private Instant deletedAt;

    @CreatedBy
    @Column(name="created_by", updatable=false)
    private UUID createdBy;

    @LastModifiedBy
    @Column(name="updated_by")
    private UUID updatedBy;
}

```

7.3 Next.js Layihə Strukturu

```

ces-service-frontend/
├── app/                                // App Router
│   ├── (auth)/
│   │   └── login/page.tsx
│   ├── (dashboard)/
│   │   ├── layout.tsx                // Sidebar, header, branch seçici
│   │   ├── page.tsx                 // Dashboard
│   │   ├── vehicles/
│   │   ├── customers/
│   │   ├── service-requests/
│   │   ├── work-orders/
│   │   ├── warehouse/
│   │   ├── inspections/
│   │   ├── reports/
│   │   └── settings/
│   └── api/                          // Next.js API routes (BFF layer)
│       └── auth/[...nextauth]/      // next-auth callback
├── components/
│   ├── ui/                          // shadcn/ui komponentləri
│   ├── layout/                      // Sidebar, Header, BranchSwitcher
│   └── modules/                     // Modul-spesifik komponentlər
├── lib/
│   └── api/                          // Axios instance, interceptors

```


SOFTWARE REQUIREMENTS SPECIFICATION

CES Service Management System

Sənəd	SRS — Part 2: Core Modules
Versiya	1.0
Tarix	29 İyun 2026
Hazırlayan	Sarkhan Babayev
Əsas	BRD v1.0 + SRS Part 1 (Foundation)

M03 Qaraj • M04 Müşəri • M05 Servis Sorğuları • M06 Work Order • M07 Xərc & Marja

M03 — Qaraj Modulu (Vehicle Management)

M03.1 Verilənlər Bazı Sxemi

vehicles cədvəli

```
CREATE TABLE ces_service.vehicles (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  garage_type        VARCHAR(50)        NOT NULL
  CHECK (garage_type IN ('COMPANY','INVESTOR','CUSTOMER')),
  owner_id           UUID                NULL, -- investors.id | customers.id
  (garage_type-a görə)
  make               VARCHAR(100)        NOT NULL, -- Marka: Caterpillar, Komatsu...
  model              VARCHAR(100)        NOT NULL, -- Model: 320D, PC200...
  year               SMALLINT            NULL,
  chassis_number     VARCHAR(100)        NULL UNIQUE,
  serial_number       VARCHAR(100)        NULL,
  plate_number        VARCHAR(50)        NULL,
  vehicle_type        VARCHAR(100)        NULL, -- Ekskavator, Buldozer...
  status              VARCHAR(50)        NOT NULL DEFAULT 'ACTIVE'
  CHECK (status IN ('ACTIVE','IN_SERVICE','INACTIVE')),
  current_location    TEXT                NULL,
  notes               TEXT                NULL,
  created_at          TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  deleted_at          TIMESTAMPTZ        NULL,
  created_by          UUID                NOT NULL REFERENCES users(id),
  updated_by          UUID                NOT NULL REFERENCES users(id)
);

CREATE INDEX idx_vehicles_branch_status ON vehicles(branch_id, status)
  WHERE deleted_at IS NULL;
CREATE INDEX idx_vehicles_garage_type ON vehicles(branch_id, garage_type)
  WHERE deleted_at IS NULL;
```

vehicle_documents cədvəli

```
CREATE TABLE ces_service.vehicle_documents (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  vehicle_id         UUID                NOT NULL REFERENCES vehicles(id) ON DELETE CASCADE,
  doc_type           VARCHAR(100)        NOT NULL, -- "Texniki Pasport", "Sigorta"...
  file_name          VARCHAR(255)        NOT NULL,
  file_url           TEXT                NOT NULL,
  file_size          BIGINT              NULL,
  expires_at         DATE                NULL,
  created_at          TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  created_by         UUID                NOT NULL REFERENCES users(id),
  updated_at          TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  updated_by         UUID                NOT NULL REFERENCES users(id),
  deleted_at          TIMESTAMPTZ        NULL
);
```

M03.2 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/vehicles	VEHICLE_READ	Siyahı (filtr + paginasıya)

Metod	URL	İcazə	Açıqlama
GET	/api/v1/vehicles/{id}	VEHICLE_READ	Tək texnika detallı
POST	/api/v1/vehicles	VEHICLE_CREATE	Yeni texnika
PUT	/api/v1/vehicles/{id}	VEHICLE_UPDATE	Yeniləmə
DELETE	/api/v1/vehicles/{id}	VEHICLE_DELETE	Soft delete
PATCH	/api/v1/vehicles/{id}/status	VEHICLE_UPDATE	Status dəyişikliyi
GET	/api/v1/vehicles/{id}/history	VEHICLE_READ	Servis tarixçəsi
POST	/api/v1/vehicles/{id}/documents	VEHICLE_UPDATE	Sənəd yükləmə
GET	/api/v1/vehicles/{id}/documents	VEHICLE_READ	Sənədlər siyahısı
DELETE	/api/v1/vehicles/{id}/documents/{docId}	VEHICLE_UPDATE	Sənəd silmə

GET /api/v1/vehicles — Query parametrləri:

Parametr	Tip	Açıqlama
garage_type	string	COMPANY INVESTOR CUSTOMER
status	string	ACTIVE IN_SERVICE INACTIVE
make	string	Marka filtri (like axtarış)
search	string	Marka, model, şassi, qeydiyyat nömrəsi üzrə
page, size, sort, dir	integer/string	Standart paginasiya

POST /api/v1/vehicles — Request body:

```
{
  "garage_type": "COMPANY",          // COMPANY | INVESTOR | CUSTOMER
  "owner_id": null,                  // investor/customer UUID (garage_type-a
  "make": "Caterpillar",
  "model": "320D",
  "year": 2019,
  "chassis_number": "CAT320D2019001",
  "serial_number": "SN-20190001",
  "plate_number": "99-AA-001",
  "vehicle_type": "Ekskavator",
  "current_location": "Bakı, Sabunçu",
  "notes": "İlk baxış 2020-ci ildə aparılıb"
}
```

Response 201:

```
{
  "success": true,
  "data": {
    "id": "550e8400-...",
  }
}
```

```
{
  "branch_id": "7c9e6679-...",
  "garage_type": "COMPANY",
  "make": "Caterpillar",
  "model": "320D",
  "year": 2019,
  "chassis_number": "CAT320D2019001",
  "status": "ACTIVE",
  "current_engine_hours": 1250.5, // M08-dən son motosaat
  "created_at": "2026-06-29T10:00:00Z"
}
```

M03.3 Biznes Məntiqi & Validasiya

- chassis_number unikal olmalıdır (bütün branch-lar üzrə)
- garage_type = INVESTOR → owner_id məcburidir (investors cədvəlindəki UUID)
- garage_type = CUSTOMER → owner_id məcburidir (customers cədvəlindəki UUID)
- garage_type = COMPANY → owner_id null ola bilər
- Status IN_SERVICE olan texnika yeni servis sorğusuna əlavə edildikdə 422 + VEHICLE_IN_SERVICE xətası qaytarılır (override mümkün — VEHICLE_OVERRIDE icazəsi tələb olunur)
- Soft delete: aktiv Work Order-i olan texnika silinə bilməz → 422 + VEHICLE_HAS_ACTIVE_WO

M04 — Müşəri İdarəetməsi

M04.1 Verilənlər Bazı Sxemi

```
CREATE TABLE ces_service.customers (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  erp_customer_id   UUID                NULL,          -- CES ERP-dəki müştəri UUID
  full_name         VARCHAR(255) NOT NULL,
  company_name      VARCHAR(255) NULL,
  voen              VARCHAR(20)  NULL,
  phone             VARCHAR(50)  NULL,
  email             VARCHAR(255) NULL,
  address           TEXT          NULL,
  customer_type     VARCHAR(50)  NOT NULL DEFAULT 'INDIVIDUAL'
                    CHECK (customer_type IN ('INDIVIDUAL', 'COMPANY')),
  is_active         BOOLEAN        NOT NULL DEFAULT TRUE,
  notes            TEXT          NULL,
  created_at        TIMESTAMPTZ     NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ     NOT NULL DEFAULT NOW(),
  deleted_at        TIMESTAMPTZ     NULL,
  created_by        UUID            NOT NULL REFERENCES users(id),
  updated_by        UUID            NOT NULL REFERENCES users(id)
);

CREATE UNIQUE INDEX idx_customers_voen ON customers(branch_id, voen)
  WHERE voen IS NOT NULL AND deleted_at IS NULL;
CREATE UNIQUE INDEX idx_customers_erp_id ON customers(erp_customer_id)
  WHERE erp_customer_id IS NOT NULL;
```

M04.2 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/customers	CUSTOMER_READ	Siyahı + axtarış + paginasıya
GET	/api/v1/customers/{id}	CUSTOMER_READ	Müşəri detalı
POST	/api/v1/customers	CUSTOMER_CREATE	Yeni müştəri (ERP-ə sinxron)
PUT	/api/v1/customers/{id}	CUSTOMER_UPDATE	Tam yeniləmə (ERP-ə sinxron)
DELETE	/api/v1/customers/{id}	CUSTOMER_DELETE	Soft delete
GET	/api/v1/customers/{id}/vehicles	CUSTOMER_READ	Müşəri texnikaları
GET	/api/v1/customers/{id}/service-history	CUSTOMER_READ	Servis tarixçəsi
POST	/api/v1/customers/sync-erp	CUSTOMER_SYNC	ERP-dən sinxronizasiya

POST /api/v1/customers — Request:

```
{
  "full_name":    "Əli Hüseynov",
  "company_name": "Hüseynov MMC",
```

```
{  "voen":      "1234567890",
  "phone":    "+994501234567",
  "email":    "ali@huseynovmmc.az",
  "address":  "Bakı, Nərimanov r., Tbilisi pr. 25",
  "customer_type": "COMPANY",
  "notes":    ""
}
```

CES ERP Sinxronizasiya axını (POST /api/v1/customers):

1. CES Service-də müştəri yaradılır
2. `ErpApiClient.createCustomer(dto)` → CES ERP /api/customers
3. ERP cavabındakı UUID → `customers.erp_customer_id`-ə yazılır
4. Əgər ERP uğursuz → müştəri CES Service-də saxlanılır, `sync_status = 'PENDING'`, `retry queue`-ya əlavə edilir

M04.3 Biznes Məntiqi

- Eyni branch daxilində eyni VÖEN ilə iki müştəri yaradıla bilməz → 409 + `DUPLICATE_VOEN`
- ERP sinxronizasiyası uğursuz olarsa sistem müştərini saxlayır, arxa planda `retry` edir (max 3 cəhd, 5 dəqiqə fasilə)
- ERP-dən sinxronlaşdırılan müştərinin `erp_customer_id` sahəsi dolu olur
- Soft delete: aktiv servis sorğusu olan müştəri silinə bilməz

M05 — Servis Sorğuları (3 növ)

Bütün servis sorğuları vahid `service_requests` cədvəlində saxlanılır. `request_type` sahəsi sorğunun növünü müəyyən edir.

M05.1 Verilənlər Bazı Sxemi

`service_requests` cədvəli

```
CREATE TABLE ces_service.service_requests (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    branch_id         UUID                NOT NULL REFERENCES branches(id),
    request_number     VARCHAR(50)        NOT NULL UNIQUE, -- SR-2026-0001
    request_type       VARCHAR(50)        NOT NULL
    CHECK (request_type IN
('PAID_SERVICE','COMPANY','INTERMEDIARY')),
    status            VARCHAR(50)        NOT NULL DEFAULT 'OPEN'
    CHECK (status IN
('OPEN','IN_PROGRESS','COMPLETED','CLOSED','CANCELLED')),
    priority           VARCHAR(20)        NOT NULL DEFAULT 'NORMAL'
    CHECK (priority IN ('NORMAL','URGENT','CRITICAL')),
    customer_id        UUID                NULL REFERENCES customers(id),
    vehicle_id         UUID                NOT NULL REFERENCES vehicles(id),
    complaint          TEXT                NOT NULL, -- Şikayət / nasazlıq təsviri
    assigned_to        UUID                NULL REFERENCES users(id),
    -- Vəsitəçilik sorğusu sahələri:
    contractor_id      UUID                NULL REFERENCES contractors(id),
    customer_amount     NUMERIC(15,2)     NULL, -- Müştəridən alınan
    contractor_amount   NUMERIC(15,2)     NULL, -- Podratçıya ödənilən
    intermediary_profit NUMERIC(15,2)     -- Hesablanmış mənfəət (generated column)
    GENERATED ALWAYS AS (customer_amount - contractor_amount) STORED,
    notes              TEXT                NULL,
    closed_at          TIMESTAMPTZ        NULL,
    created_at         TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
    updated_at         TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
    deleted_at         TIMESTAMPTZ        NULL,
    created_by         UUID                NOT NULL REFERENCES users(id),
    updated_by         UUID                NOT NULL REFERENCES users(id)
);

-- Avtomatik nömrələmə funksiyası
CREATE SEQUENCE ces_service.sr_number_seq START 1;

CREATE OR REPLACE FUNCTION generate_sr_number()
RETURNS TRIGGER AS $$
BEGIN
    NEW.request_number := 'SR-' || TO_CHAR(NOW(),'YYYY') || '-'
    || LPAD(nextval('ces_service.sr_number_seq')::TEXT, 4, '0');
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_sr_number
BEFORE INSERT ON service_requests
FOR EACH ROW EXECUTE FUNCTION generate_sr_number();
```

`service_request_services` — Xidmət siyahısı (Pullu Xidmət)

```
CREATE TABLE ces_service.service_request_services (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    branch_id         UUID                NOT NULL REFERENCES branches(id),
    service_request_id UUID                NOT NULL REFERENCES service_requests(id) ON
DELETE CASCADE,
    service_name       VARCHAR(255)      NOT NULL,
    pricing_type       VARCHAR(20)       NOT NULL
```

```

        CHECK (pricing_type IN ('FIXED', 'HOURLY')),
        unit_price      NUMERIC(15,2) NOT NULL,    -- Saatlıq qiymət və ya sabit qiymət
        hours_worked    NUMERIC(5,2)  NULL,        -- Yalnız HOURLY üçün
        total_price      NUMERIC(15,2) NOT NULL,    -- FIXED: unit_price | HOURLY:
        unit_price×hours
        notes           TEXT              NULL,
        created_at      TIMESTAMPTZ      NOT NULL DEFAULT NOW(),
        updated_at      TIMESTAMPTZ      NOT NULL DEFAULT NOW(),
        created_by      UUID              NOT NULL REFERENCES users(id),
        updated_by      UUID              NOT NULL REFERENCES users(id)
    );

```

service_request_tasks — Şirkət sorğusu dinamik tapşırıqları

```

CREATE TABLE ces_service.service_request_tasks (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    branch_id         UUID                NOT NULL REFERENCES branches(id),
    service_request_id UUID              NOT NULL REFERENCES service_requests(id) ON
DELETE CASCADE,
    task_description  TEXT                NOT NULL,
    assigned_to       UUID                NULL REFERENCES users(id),
    status            VARCHAR(50)         NOT NULL DEFAULT 'OPEN'
    CHECK (status IN ('OPEN', 'IN_PROGRESS', 'COMPLETED')),
    started_at        TIMESTAMPTZ        NULL,
    completed_at      TIMESTAMPTZ        NULL,
    notes             TEXT                NULL,
    created_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
    created_by        UUID                NOT NULL REFERENCES users(id),
    updated_by        UUID                NOT NULL REFERENCES users(id)
);

```

contractors — Podratçılar (Vasitəçilik üçün)

```

CREATE TABLE ces_service.contractors (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    branch_id         UUID                NOT NULL REFERENCES branches(id),
    name              VARCHAR(255)        NOT NULL,
    phone             VARCHAR(50)         NULL,
    email             VARCHAR(255)        NULL,
    address           TEXT                NULL,
    voen              VARCHAR(20)         NULL,
    is_active         BOOLEAN              NOT NULL DEFAULT TRUE,
    notes             TEXT                NULL,
    created_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
    deleted_at        TIMESTAMPTZ        NULL,
    created_by        UUID                NOT NULL REFERENCES users(id),
    updated_by        UUID                NOT NULL REFERENCES users(id)
);

```

M05.2 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/service-requests	SR_READ	Siyahı — tip/status/tarix filtri
GET	/api/v1/service-requests/{id}	SR_READ	Tam detallar + xidmətlər + tapşırıqlar
POST	/api/v1/service-requests	SR_CREATE	Yeni sorğu (tip-ə görə payload fərqlənir)

Metod	URL	İcazə	Açıqlama
PUT	/api/v1/service-requests/{id}	SR_UPDATE	Əsas məlumatları yeniləmə
PATCH	/api/v1/service-requests/{id}/status	SR_UPDATE	Status dəyişikliyi
DELETE	/api/v1/service-requests/{id}	SR_DELETE	Soft delete (yalnız OPEN statusda)
POST	/api/v1/service-requests/{id}/services	SR_UPDATE	Xidmət əlavəsi (PAID_SERVICE)
PUT	/api/v1/service-requests/{id}/services/{sld}	SR_UPDATE	Xidmət yeniləmə
DELETE	/api/v1/service-requests/{id}/services/{sld}	SR_UPDATE	Xidmət silmə
POST	/api/v1/service-requests/{id}/tasks	SR_UPDATE	Tapşırıq əlavəsi (COMPANY)
PATCH	/api/v1/service-requests/{id}/tasks/{tld}	SR_UPDATE	Tapşırıq status/məlumat yeniləmə
PATCH	/api/v1/service-requests/{id}/intermediary	SR_UPDATE	Vasitəçilik məbləğlərini yeniləmə
GET	/api/v1/contractors	CONTRACTOR_READ	Podratçılar siyahısı
POST	/api/v1/contractors	CONTRACTOR_CREATE	Yeni podratçı

POST /api/v1/service-requests — PAID_SERVICE:

```
{
  "request_type": "PAID_SERVICE",
  "priority": "NORMAL",
  "customer_id": "uuid-of-customer",
  "vehicle_id": "uuid-of-vehicle",
  "complaint": "Mühərrik yağı sızıntısı var",
  "assigned_to": "uuid-of-mechanic",
  "services": [
    {
      "service_name": "Yağ dəyişimi",
      "pricing_type": "HOURLY",
      "unit_price": 25.00,
      "hours_worked": 2.5
    },
    {
      "service_name": "Conta dəyişimi",
      "pricing_type": "FIXED",
      "unit_price": 80.00
    }
  ]
}
```

POST /api/v1/service-requests — COMPANY:

```
{
  "request_type": "COMPANY",
  "priority": "URGENT",
  "vehicle_id": "uuid-of-company-vehicle",
  "complaint": "Hidravlik sistem problemi",
  "tasks": [
    {
      "task_description": "Hidravlik yağ səviyyəsini yoxla",
      "assigned_to": "uuid-of-mechanic"
    }
  ]
}
```

}

POST /api/v1/service-requests — INTERMEDIARY:

```
{
  "request_type":      "INTERMEDIARY",
  "priority":          "NORMAL",
  "customer_id":       "uuid-of-customer",
  "vehicle_id":        "uuid-of-vehicle",
  "complaint":         "Transmissiya problemi",
  "contractor_id":     "uuid-of-contractor",
  "customer_amount":   100.00,
  "contractor_amount": 60.00
}
```

intermediary_profit = customer_amount - contractor_amount PostgreSQL GENERATED ALWAYS AS sütunu ilə avtomatik hesablanır. Java tərəfindən hesablamaq lazım deyil.

M05.3 Biznes Məntiqi & Validasiya

Kod	Qayda
SR-V01	PAID_SERVICE: customer_id məcburidir
SR-V02	PAID_SERVICE: ən azı bir service əlavə olunmadan sorğu CLOSED statusuna keçə bilməz → 422 + SR_NO_SERVICES
SR-V03	HOURLY xidmətdə hours_worked > 0 olmalıdır → 422 + SR_INVALID_HOURS
SR-V04	COMPANY: vehicle_id COMPANY və ya INVESTOR qarajından olmalıdır → 422 + SR_INVALID_VEHICLE_TYPE
SR-V05	INTERMEDIARY: contractor_id, customer_amount, contractor_amount məcburidir
SR-V06	INTERMEDIARY: contractor_amount ≤ customer_amount olmalıdır → 422 + SR_CONTRACTOR_AMOUNT_EXCEEDS
SR-V07	COMPANY sorğusuna yeni tapşırıq sorğu OPEN və ya IN_PROGRESS statusunda olanda əlavə edilə bilər
SR-V08	Sorğu CLOSED statusuna keçdikdə Work Order da CLOSED olmalıdır (M06 ilə sinxron)
SR-V09	CANCELLED sorğu yenidən açıla bilməz — yeni sorğu yaratmaq lazımdır

M06 — Work Order İdarəetməsi

Hər servis sorğusu üçün avtomatik Work Order yaradılır. Work Order texniki icra sənədidir — sorğunun status axınıni, tapşırıq tarixçesini və bağlanma prosesini idarə edir.

M06.1 Verilənlər Bazı Sxemi

work_orders cədvəli

```
CREATE TABLE ces_service.work_orders (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  wo_number          VARCHAR(50)        NOT NULL UNIQUE, -- WO-2026-0001
  service_request_id UUID                NOT NULL REFERENCES service_requests(id),
  wo_type            VARCHAR(50)        NOT NULL
  CHECK (wo_type IN
    ('PAID_SERVICE', 'COMPANY', 'INTERMEDIARY', 'INSPECTION')),
  status             VARCHAR(50)        NOT NULL DEFAULT 'NEW'
  CHECK (status IN
    ('NEW', 'ASSIGNED', 'IN_PROGRESS', 'ON_HOLD', 'COMPLETED', 'CLOSED', 'CANCELLED')),
  priority            VARCHAR(20)        NOT NULL DEFAULT 'NORMAL'
  CHECK (priority IN ('NORMAL', 'URGENT', 'CRITICAL')),
  assigned_to         UUID                NULL REFERENCES users(id),
  assigned_at         TIMESTAMPTZ        NULL,
  started_at          TIMESTAMPTZ        NULL,
  completed_at         TIMESTAMPTZ        NULL,
  closed_at           TIMESTAMPTZ        NULL,
  closed_by           UUID                NULL REFERENCES users(id),
  total_cost           NUMERIC(15,2)     NULL, -- Ümumi xərc (daxili)
  total_sell           NUMERIC(15,2)     NULL, -- Müştəriyə göstərilən məbləğ
  is_cost_approved     BOOLEAN            NOT NULL DEFAULT FALSE,
  cost_approved_by     UUID                NULL REFERENCES users(id),
  cost_approved_at     TIMESTAMPTZ        NULL,
  closure_notes        TEXT              NULL,
  created_at           TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  updated_at           TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  deleted_at           TIMESTAMPTZ        NULL,
  created_by           UUID                NOT NULL REFERENCES users(id),
  updated_by           UUID                NOT NULL REFERENCES users(id)
);

CREATE SEQUENCE ces_service.wo_number_seq START 1;
-- WO nömrəsi üçün eyni trigger məntiqi SR ilə eynidir (WO-2026-XXXX)
```

work_order_status_history — Status tarixçəsi

```
CREATE TABLE ces_service.work_order_status_history (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  wo_id             UUID                NOT NULL REFERENCES work_orders(id) ON DELETE
  CASCADE,
  from_status        VARCHAR(50)        NULL, -- NULL = ilk status
  to_status          VARCHAR(50)        NOT NULL,
  changed_by         UUID                NOT NULL REFERENCES users(id),
  changed_at         TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  notes              TEXT              NULL
);
```

work_order_comments — Şərhlər

```
CREATE TABLE ces_service.work_order_comments (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
```

```

    wo_id        UUID        NOT NULL REFERENCES work_orders(id) ON DELETE CASCADE,
    comment      TEXT        NOT NULL,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    created_by   UUID        NOT NULL REFERENCES users(id),
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_by   UUID        NOT NULL REFERENCES users(id)
);

```

work_order_attachments — Fayllar

```

CREATE TABLE ces_service.work_order_attachments (
    id            UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    branch_id     UUID        NOT NULL REFERENCES branches(id),
    wo_id         UUID        NOT NULL REFERENCES work_orders(id) ON DELETE CASCADE,
    file_name     VARCHAR(255) NOT NULL,
    file_url      TEXT        NOT NULL,
    file_type     VARCHAR(100) NULL, -- image/jpeg, application/pdf...
    file_size     BIGINT      NULL,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    created_by    UUID        NOT NULL REFERENCES users(id),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_by    UUID        NOT NULL REFERENCES users(id)
);

```

M06.2 Status Keçid Matrisi

Status keçidi yalnız aşağıdakı cədvəldəki sətirlərə uyğun olarsa qəbul edilir. Digər keçid cəhdləri 422 + WO_INVALID_STATUS_TRANSITION qaytarır.

Cari Status	İcazəli Növbəti	Tələb olunan İcazə	Şərt
NEW	ASSIGNED	WO_UPDATE	assigned_to doldurulmalıdır
NEW	CANCELLED	WO_CANCEL	Əsas qeyd məcburidir
ASSIGNED	IN_PROGRESS	WO_UPDATE	—
ASSIGNED	CANCELLED	WO_CANCEL	Əsas qeyd məcburidir
IN_PROGRESS	ON_HOLD	WO_UPDATE	Səbəb qeyd edilməlidir
IN_PROGRESS	COMPLETED	WO_UPDATE	closure_notes məcburi
ON_HOLD	IN_PROGRESS	WO_UPDATE	—
ON_HOLD	CANCELLED	WO_CANCEL	Əsas qeyd məcburidir
COMPLETED	CLOSED	WO_CLOSE + APPROVE_COST	is_cost_approved = true olmalıdır
COMPLETED	IN_PROGRESS	WO_UPDATE	Yenidən işə qaytarma

M06.3 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/work-orders	WO_READ	Siyahı + filtr + paginasıya
GET	/api/v1/work-orders/{id}	WO_READ	Tam WO detalı
POST	/api/v1/work-orders	WO_CREATE	Manuel WO yaratma (adətən avtomatik)

Metod	URL	İcazə	Açıqlama
PATCH	/api/v1/work-orders/{id}/status	WO_UPDATE	Status keçidi
PATCH	/api/v1/work-orders/{id}/assign	WO_UPDATE	Mexanik təyinatı
PATCH	/api/v1/work-orders/{id}/close	WO_CLOSE	WO bağlama (COMPLETED→CLOSED)
PATCH	/api/v1/work-orders/{id}/approve-cost	APPROVE_COST	Xərc təsdiqi
GET	/api/v1/work-orders/{id}/history	WO_READ	Status tarixçəsi
POST	/api/v1/work-orders/{id}/comments	WO_UPDATE	Şərh əlavəsi
GET	/api/v1/work-orders/{id}/comments	WO_READ	Şərhlər siyahısı
POST	/api/v1/work-orders/{id}/attachments	WO_UPDATE	Fayl yükləmə
GET	/api/v1/work-orders/{id}/attachments	WO_READ	Fayllar siyahısı

PATCH /api/v1/work-orders/{id}/status — Request:

```
{
  "status": "IN_PROGRESS",
  "notes": "İşə başlanıldı – mühərrik söküldü"
}
```

PATCH /api/v1/work-orders/{id}/close — Request:

```
{
  "closure_notes": "Yağ süzgəci dəyişdirildi, sızıntı aradan qaldırıldı",
  "total_cost": 142.50,
  "total_sell": 185.00
}
```

WO CLOSED statusuna keçdikdə sistem avtomatik: 1) Sorğunu arxivə köçürür, 2) CES ERP mühasibatlıq moduluna maliyyə məlumatlarını push edir, 3) Anbar stok çıxışlarını təsdiqlənmiş kimi işarələyir.

M06.4 Avtomatik WO Yaradılması

- Servis sorğusu yaradıldıqda (POST /service-requests) sistem avtomatik Work Order yaradır
- Texniki baxışda ciddi problem tapıldıqda M18 modulu avtomatik Work Order yaradır
- Profilaktik servis planı tetiklendiikdə avtomatik Work Order yaradılır (gələcək Faza 2)
- Avtomatik yaradılan WO-ların wo_type = sorğunun request_type dəyərinə bərabərdir

M07 — Xərc & Marja İdarəetməsi

Bu modul sistemin maliyyə nüvəsidir. Hər Work Order üçün detal xərcləri, işçilik xərcləri və marja hesablamaları idarə olunur. Anbar modulu (M19) ilə tam inteqrasiyalıdır.

M07.1 Verilənlər Bazı Sxemi

wo_cost_items — Work Order xərc maddələri

```
CREATE TABLE ces_service.wo_cost_items (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  wo_id             UUID                NOT NULL REFERENCES work_orders(id) ON DELETE
  CASCADE,
  item_type         VARCHAR(50) NOT NULL
  CHECK (item_type IN
('LABOR', 'PART', 'EXTERNAL_SERVICE', 'OTHER')),
  -- PART üçün anbar bağlantısı:
  spare_part_id     UUID                NULL REFERENCES spare_parts(id),
  spare_part_usage_id UUID            NULL REFERENCES spare_part_usages(id),
  description       VARCHAR(255) NOT NULL,
  quantity          NUMERIC(10,3) NOT NULL DEFAULT 1,
  unit_cost         NUMERIC(15,2) NOT NULL, -- Maya dəyəri
  margin_percent    NUMERIC(5,2) NOT NULL DEFAULT 0,
  unit_sell_price   NUMERIC(15,2) NOT NULL, -- Maya × (1 + margin/100)
  total_cost        NUMERIC(15,2) NOT NULL, -- quantity × unit_cost
  total_sell_price  NUMERIC(15,2) NOT NULL, -- quantity × unit_sell_price
  is_margin_overridden BOOLEAN NOT NULL DEFAULT FALSE,
  margin_override_by UUID            NULL REFERENCES users(id),
  margin_override_at TIMESTAMPTZ NULL,
  notes            TEXT              NULL,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  created_by       UUID            NOT NULL REFERENCES users(id),
  updated_by       UUID            NOT NULL REFERENCES users(id)
);
```

margin_config — Sistem default marja konfigurasiyası

```
CREATE TABLE ces_service.margin_config (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  item_type         VARCHAR(50) NOT NULL, -- LABOR | PART | EXTERNAL_SERVICE
  default_margin    NUMERIC(5,2) NOT NULL DEFAULT 0,
  min_margin        NUMERIC(5,2) NOT NULL DEFAULT 0,
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by        UUID            NOT NULL REFERENCES users(id),
  UNIQUE (branch_id, item_type)
);
```

M07.2 Hesablama Məntiqi

Bütün hesablamalar Java service katmanında BigDecimal ilə aparılır. DB-də NUMERIC(15,2) saxlanılır.

Hesablama	Formula
Vahid satış qiyməti	$\text{unit_sell_price} = \text{unit_cost} \times (1 + \text{margin_percent} / 100)$
Ümumi xərc	$\text{total_cost} = \text{quantity} \times \text{unit_cost}$

Hesablama	Formula
Ümumi satış	$\text{total_sell_price} = \text{quantity} \times \text{unit_sell_price}$
WO ümumi xərci	SUM(total_cost) bütün wo_cost_items üzrə
WO ümumi satışı	SUM(total_sell_price) bütün wo_cost_items üzrə + xidmət haqları
WO mənfəəti	$\text{total_sell} - \text{total_cost}$
Vasitəçilik mənfəəti	customer_amount - contractor_amount (DB-də generated)

Java service kodunda hesablama nümunəsi:

```
public WoCostItemResponse addCostItem(UUID woId, WoCostItemRequest req) {
    BigDecimal unitSell = req.getUnitCost()
        .multiply(BigDecimal.ONE.add(
            req.getMarginPercent().divide(new BigDecimal("100"), 4, HALF_UP)));
    BigDecimal totalCost = req.getUnitCost().multiply(req.getQuantity());
    BigDecimal totalSell = unitSell.multiply(req.getQuantity());
    // ...entity save...
    updateWoTotals(woId); // WO cəmini yenilə
}
```

M07.3 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/work-orders/{id}/cost-items	WO_READ	WO xərc maddələri siyahısı
POST	/api/v1/work-orders/{id}/cost-items	WO_UPDATE	Xərc maddəsi əlavəsi
PUT	/api/v1/work-orders/{id}/cost-items/{itemId}	WO_UPDATE	Xərc maddəsi yeniləmə
DELETE	/api/v1/work-orders/{id}/cost-items/{itemId}	WO_UPDATE	Xərc maddəsi silmə
PATCH	/api/v1/work-orders/{id}/cost-items/{itemId}/margin	CHANGE_MARGIN	Marja dəyişikliyi
GET	/api/v1/work-orders/{id}/cost-summary	WO_READ	WO maliyyə xülasəsi
GET	/api/v1/margin-config	MARGIN_CONFIG_READ	Sistem default marjaları
PUT	/api/v1/margin-config/{itemType}	MARGIN_CONFIG_UPDATE	Default marja yeniləmə

POST /api/v1/work-orders/{id}/cost-items — Request (PART tipi):

```
{
  "item_type": "PART",
  "spare_part_id": "uuid-of-spare-part", // M19 anbarından
  "description": "Yağ süzgəci Mann W940",
  "quantity": 2,
  "unit_cost": 12.50, // Anbardan avtomatik gəlir
  "margin_percent": 25.00 // Anbarın default marjası, dəyişdirilə bilər
}
```

POST /api/v1/work-orders/{id}/cost-items — Request (LABOR tipi):

```
{
  "item_type":      "LABOR",
  "description":    "Mexanik işçilik — Yağ dəyişimi",
  "quantity":       2.5,      // Saat
  "unit_cost":      20.00,    // Saatlıq işçilik xərci
  "margin_percent": 30.00
}
```

GET /api/v1/work-orders/{id}/cost-summary — Response:

```
{
  "success": true,
  "data": {
    "wo_id":      "...",
    "wo_number":  "WO-2026-0042",
    "items_breakdown": {
      "labor":      { "total_cost": 50.00, "total_sell": 65.00 },
      "parts":      { "total_cost": 25.00, "total_sell": 31.25 },
      "external_service": { "total_cost": 0.00, "total_sell": 0.00 }
    },
    "service_charges": 87.50, // M05-dəki xidmət haqları
    "grand_total_cost": 75.00,
    "grand_total_sell": 183.75,
    "total_profit": 108.75,
    "profit_margin_pct": 59.18,
    "is_cost_approved": false
  }
}
```

M07.4 Biznes Qaydaları & Validasiya

Kod	Qayda
CM-V01	unit_sell_price < unit_cost → 422 + COST_BELOW_UNIT_COST (xəbərdarlıq — icazəli istifadəçi override edə bilər)
CM-V02	margin_percent < margin_config.min_margin → 422 + MARGIN_BELOW_MINIMUM
CM-V03	Marja dəyişikliyi yalnız CHANGE_MARGIN icazəsi ilə → 403 + PERMISSION_DENIED
CM-V04	Marja dəyişikliyi Audit Log-a yazılır: əvvəlki dəyər, yeni dəyər, kim, nə vaxt
CM-V05	WO CLOSED statusunda xərc maddəsi əlavə/dəyişiklik edilə bilməz → 422 + WO_CLOSED
CM-V06	PART tipli xərc maddəsi əlavə edildikdə M19 anbarından stok azalır (spare_part_usages yazılır)
CM-V07	Stok çatışmırsa PART əlavəsi qəbul edilmir → 422 + STOCK_INSUFFICIENT
CM-V08	quantity ≤ 0 → 400 + INVALID_QUANTITY

SOFTWARE REQUIREMENTS SPECIFICATION

CES Service Management System

Sənəd	SRS — Part 3: Service Operations
Versiya	1.0
Tarix	29 İyun 2026
Hazırlayan	Sarkhan Babayev
Əsas	BRD v1.0 + SRS Part 1 & Part 2

M08 Motosaat • M18 Texniki Baxış • M19 Anbar & Ehtiyat Hissələri

M08 — Motosaat İdarəetməsi (Engine Hours)

Texnikaların mühərrik saatlarının izlənməsi. Şirkət Qarajındakı texnikalar üçün kritik — həm Work Order xərc hesablamasının, həm Texniki Baxış triggerinin əsasını təşkil edir.

M08.1 Verilənlər Bazı Sxemi

engine_hour_logs — Motosaat qeyd tarixçəsi

```
CREATE TABLE ces_service.engine_hour_logs (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  vehicle_id        UUID                NOT NULL REFERENCES vehicles(id),
  hours_value        NUMERIC(10,1) NOT NULL,          -- Yeni motosaat dəyəri
  previous_value     NUMERIC(10,1) NULL,             -- Əvvəlki qeyd (audit üçün)
  delta             NUMERIC(10,1)          -- Artım (generated)
  GENERATED ALWAYS AS (hours_value - previous_value) STORED,
  entry_type         VARCHAR(50)         NOT NULL DEFAULT 'MANUAL'
  CHECK (entry_type IN ('MANUAL', 'MOBILE', 'INSPECTION')),
  -- Azalan dəyər hallı (sayğac dəyişməsi):
  is_rollover        BOOLEAN             NOT NULL DEFAULT FALSE,
  rollover_reason    TEXT                NULL,
  source_ref_id      UUID                NULL, -- inspection_id və ya wo_id (gəldiyi yer)
  recorded_at        DATE                NOT NULL DEFAULT CURRENT_DATE,
  notes              TEXT                NULL,
  created_at         TIMESTAMPTZ         NOT NULL DEFAULT NOW(),
  created_by         UUID                NOT NULL REFERENCES users(id),
  updated_at         TIMESTAMPTZ         NOT NULL DEFAULT NOW(),
  updated_by         UUID                NOT NULL REFERENCES users(id)
);

CREATE INDEX idx_eh_logs_vehicle_date
  ON engine_hour_logs(vehicle_id, recorded_at DESC);
```

engine_hour_alerts — Motosaat xəbərdarlıq konfigurasiyası

```
CREATE TABLE ces_service.engine_hour_alerts (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  vehicle_id        UUID                NOT NULL REFERENCES vehicles(id),
  alert_type         VARCHAR(50)         NOT NULL
  CHECK (alert_type IN ('INSPECTION_DUE', 'CUSTOM')),
  threshold_hours    NUMERIC(10,1) NOT NULL, -- Bu saata çatdıqda xəbərdarlıq
  warning_before     NUMERIC(10,1) NOT NULL DEFAULT 50, -- X saat əvvəl xəbərdar et
  is_active          BOOLEAN             NOT NULL DEFAULT TRUE,
  last_triggered_at   TIMESTAMPTZ        NULL,
  created_at         TIMESTAMPTZ         NOT NULL DEFAULT NOW(),
  updated_at         TIMESTAMPTZ         NOT NULL DEFAULT NOW(),
  created_by         UUID                NOT NULL REFERENCES users(id),
  updated_by         UUID                NOT NULL REFERENCES users(id)
);
```

vehicles.current_engine_hours: Sürətli oxuma üçün vehicles cədvəlində ayrıca saxlanılır. Hər yeni engine_hour_logs yazısından sonra trigger ilə yenilənir.

vehicles cədvəlinə əlavə sütun (migration ilə)

```
-- V3__add_engine_hours_to_vehicles.sql
ALTER TABLE ces_service.vehicles
  ADD COLUMN current_engine_hours NUMERIC(10,1) NULL DEFAULT 0,
  ADD COLUMN last_engine_hours_at TIMESTAMPTZ    NULL;

-- Trigger: engine_hour_logs INSERT-dən sonra vehicles-i yenilə
```

```

CREATE OR REPLACE FUNCTION sync_vehicle_engine_hours()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE ces_service.vehicles
    SET     current_engine_hours = NEW.hours_value,
           last_engine_hours_at = NEW.created_at
    WHERE  id = NEW.vehicle_id;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_sync_engine_hours
AFTER INSERT ON engine_hour_logs
FOR EACH ROW EXECUTE FUNCTION sync_vehicle_engine_hours();

```

M08.2 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/vehicles/{id}/engine-hours	EH_READ	Motosaat tarixçəsi (paginasıya)
POST	/api/v1/vehicles/{id}/engine-hours	EH_CREATE	Yeni motosaat qeydiyyatı
GET	/api/v1/vehicles/{id}/engine-hours/latest	EH_READ	Son motosaat dəyəri
GET	/api/v1/vehicles/{id}/engine-hours/alerts	EH_READ	Aktiv xəbərdarlıqlar
POST	/api/v1/vehicles/{id}/engine-hours/alerts	EH_ALERT_MANAGE	Xəbərdarlıq yaratma
PUT	/api/v1/vehicles/{id}/engine-hours/alerts/{ald}	EH_ALERT_MANAGE	Xəbərdarlıq yeniləmə
DELETE	/api/v1/vehicles/{id}/engine-hours/alerts/{ald}	EH_ALERT_MANAGE	Xəbərdarlıq silmə
GET	/api/v1/engine-hours/overdue	EH_READ	Baxış vaxtı keçmiş texnikalar (branch üzrə)

POST /api/v1/vehicles/{id}/engine-hours — Request:

```

{
  "hours_value": 1350.5,
  "entry_type": "MANUAL", // MANUAL | MOBILE | INSPECTION
  "recorded_at": "2026-06-29",
  "notes": "Aylıq baxış zamanı qeyd edildi",
  "is_rollover": false
}

```

Response 201:

```

{
  "success": true,
  "data": {
    "id": "uuid",
    "vehicle_id": "uuid",
    "hours_value": 1350.5,
    "previous_value": 1250.0,
    "delta": 100.5,
    "entry_type": "MANUAL",
    "is_rollover": false,
    "recorded_at": "2026-06-29",
    "alerts_triggered": [ // Bu qeyd ilə tetiklənən xəbərdarlıqlar
      {

```

```

    "alert_type": "INSPECTION_DUE",
    "threshold": 1250.0,
    "message": "Texniki baxış vaxtı keçib"
  }
]
}
}

```

M08.3 Biznes Məntiqi & Validasiya

Kod	Qayda
EH-V01	hours_value < vehicles.current_engine_hours AND is_rollover = false → 422 + ENGINE_HOURS_DECREASING
EH-V02	is_rollover = true olduqda rollover_reason məcburidir → 400 + ROLLOVER_REASON_REQUIRED
EH-V03	hours_value > 99999.9 → 400 + ENGINE_HOURS_OUT_OF_RANGE
EH-V04	recorded_at gələcək tarix ola bilməz → 400 + FUTURE_DATE_NOT_ALLOWED
EH-V05	Hər yeni qeyddən sonra aktiv xəbərdarlıqlar yoxlanılır — threshold keçilmişsə bildiriş göndərilir
EH-V06	M18 Texniki Baxış formasından gələn qeydlər entry_type = INSPECTION, source_ref_id = inspection_id
EH-V07	Mobil tətbiqdən gələn qeydlər entry_type = MOBILE — eyni validasiya qaydaları tətbiq olunur

M08.4 Xəbərdarlıq Tetiklənmə Məntiqi

Hər yeni motosaat qeydi POST edildə Service katmanı aşağıdakı yoxlamaları aparır:

```

// EngineHourService.java
void checkAndTriggerAlerts(UUID vehicleId, BigDecimal newHours) {
    List<EngineHourAlert> alerts = alertRepo
        .findActiveByVehicleId(vehicleId);

    for (EngineHourAlert alert : alerts) {
        // Xəbərdarlıq həddinə çatdı mı?
        boolean thresholdReached =
            newHours.compareTo(alert.getThresholdHours()) >= 0;
        // Əvvəlcədən xəbərdar etmi?
        boolean warningZone = newHours.compareTo(
            alert.getThresholdHours().subtract(alert.getWarningBefore())) >= 0;

        if (thresholdReached || warningZone) {
            notificationService.send(alert, newHours);
            alert.setLastTriggeredAt(Instant.now());
            alertRepo.save(alert);
        }
    }
}

```


M18 — Texniki Baxış (Preventive Inspection)

Şirkət Qarajındakı texnikaların (COMPANY + INVESTOR) dövri texniki baxışının idarə olunması. Baxış nəticəsinə görə ya texnika Aktiv qalır, ya da avtomatik Work Order yaradılır.

M18.1 Verilənlər Bazı Sxemi

inspection_schedules — Baxış planı konfigurasiyası

```
CREATE TABLE ces_service.inspection_schedules (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  vehicle_id        UUID                NOT NULL REFERENCES vehicles(id),
  trigger_type       VARCHAR(50)        NOT NULL
                                CHECK (trigger_type IN ('HOURS', 'DAYS', 'BOTH')),
  hours_interval     NUMERIC(10,1)      NULL,    -- Hər X motosaatda (HOURS | BOTH)
  days_interval      INTEGER            NULL,    -- Hər X gündə (DAYS | BOTH)
  next_due_hours     NUMERIC(10,1)      NULL,    -- Növbəti baxış motosaat həddi
  next_due_date      DATE               NULL,    -- Növbəti baxış tarixi
  warning_before_hours NUMERIC(10,1)    NOT NULL DEFAULT 50,
  warning_before_days INTEGER          NOT NULL DEFAULT 7,
  is_active          BOOLEAN            NOT NULL DEFAULT TRUE,
  created_at         TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
  updated_at         TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
  deleted_at         TIMESTAMPTZ       NULL,
  created_by         UUID               NOT NULL REFERENCES users(id),
  updated_by         UUID               NOT NULL REFERENCES users(id),
  UNIQUE (vehicle_id, branch_id)      -- Bir texnika üçün bir plan
);
```

inspection_checklists — Baxış siyahısı şablonları

```
CREATE TABLE ces_service.inspection_checklists (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  name              VARCHAR(255)        NOT NULL,  -- "Standart Aylıq Baxış"
  description       TEXT                NULL,
  is_default        BOOLEAN            NOT NULL DEFAULT FALSE,
  is_active         BOOLEAN            NOT NULL DEFAULT TRUE,
  created_at        TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
  created_by        UUID               NOT NULL REFERENCES users(id),
  updated_by        UUID               NOT NULL REFERENCES users(id)
);
```

inspection_checklist_items — Yoxlama maddələri şablonu

```
CREATE TABLE ces_service.inspection_checklist_items (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  checklist_id       UUID                NOT NULL REFERENCES inspection_checklists(id) ON
DELETE CASCADE,
  item_name         VARCHAR(255)        NOT NULL,  -- "Mühərrik yağı"
  item_category     VARCHAR(100)        NULL,    -- "Yağlama sistemi"
  options           JSONB               NOT NULL,  -- ["Normal", "Az", "Çox", "Çirkli"]
  requires_value     BOOLEAN            NOT NULL DEFAULT FALSE,  -- Rəqəm sahəsi?
  value_unit        VARCHAR(50)         NULL,    -- "saat", "km", "bar"
  sort_order        INTEGER            NOT NULL DEFAULT 0,
  is_active         BOOLEAN            NOT NULL DEFAULT TRUE,
  created_at        TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
  created_by        UUID               NOT NULL REFERENCES users(id),
  updated_by        UUID               NOT NULL REFERENCES users(id)
);
```

inspections — Baxış qeydləri

```
CREATE TABLE ces_service.inspections (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  inspection_number VARCHAR(50)         NOT NULL UNIQUE, -- INS-2026-0001
  vehicle_id        UUID                NOT NULL REFERENCES vehicles(id),
  checklist_id       UUID                NOT NULL REFERENCES inspection_checklists(id),
  inspector_id       UUID                NOT NULL REFERENCES users(id),
  status            VARCHAR(50)         NOT NULL DEFAULT 'IN_PROGRESS'
  CHECK (status IN ('IN_PROGRESS', 'COMPLETED', 'CANCELLED')),
  overall_result     VARCHAR(50)        NULL
  CHECK (overall_result IN
('OK', 'MINOR_ISSUE', 'CRITICAL_ISSUE')),
  engine_hours_at_inspection NUMERIC(10,1) NULL, -- Baxış anındakı motosaat
  inspection_date    DATE                NOT NULL DEFAULT CURRENT_DATE,
  completed_at       TIMESTAMPTZ        NULL,
  notes             TEXT                 NULL,
  -- Avtomatik yaradılan WO bağlantısı:
  generated_wo_id    UUID                NULL REFERENCES work_orders(id),
  created_at         TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  updated_at         TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  deleted_at         TIMESTAMPTZ        NULL,
  created_by         UUID                NOT NULL REFERENCES users(id),
  updated_by         UUID                NOT NULL REFERENCES users(id)
);
```

inspection_results — Hər maddənin nəticəsi

```
CREATE TABLE ces_service.inspection_results (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  inspection_id      UUID                NOT NULL REFERENCES inspections(id) ON DELETE
  CASCADE,
  checklist_item_id  UUID                NOT NULL REFERENCES
  inspection_checklist_items(id),
  selected_option    VARCHAR(100)       NOT NULL, -- "Normal", "Az", "Dəyişdirmə lazım"
  numeric_value      NUMERIC(10,2)      NULL, -- requires_value = true olduqda
  severity           VARCHAR(50)         NOT NULL DEFAULT 'OK'
  CHECK (severity IN ('OK', 'WARNING', 'CRITICAL')),
  notes             TEXT                 NULL,
  created_at         TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  updated_at         TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  created_by         UUID                NOT NULL REFERENCES users(id),
  updated_by         UUID                NOT NULL REFERENCES users(id)
);
```

M18.2 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/inspection-schedules	INSP_READ	Bütün planlar (filtr: vehicle, status)
POST	/api/v1/inspection-schedules	INSP_MAN AGE	Texnika üçün baxış planı yaratma
PUT	/api/v1/inspection-schedules/{id}	INSP_MAN AGE	Plan yeniləmə (interval, trigger)
DELETE	/api/v1/inspection-schedules/{id}	INSP_MAN AGE	Plan silmə

Metod	URL	İcazə	Açıqlama
GET	/api/v1/inspection-checklists	INSP_READ	Checklist şablonları siyahısı
POST	/api/v1/inspection-checklists	INSP_MAN AGE	Yeni checklist şablonu
POST	/api/v1/inspection-checklists/{id}/items	INSP_MAN AGE	Şablona maddə əlavəsi
PUT	/api/v1/inspection-checklists/{id}/items/{itemId}	INSP_MAN AGE	Maddə yeniləmə
DELETE	/api/v1/inspection-checklists/{id}/items/{itemId}	INSP_MAN AGE	Maddə silmə
GET	/api/v1/inspections	INSP_READ	Baxış qeydləri (filtr + paginasıya)
GET	/api/v1/inspections/{id}	INSP_READ	Baxış detallı + nəticələr
POST	/api/v1/inspections	INSP_CREA TE	Yeni baxış başlatma
POST	/api/v1/inspections/{id}/results	INSP_UPDA TE	Maddə nəticələrini daxil etmə
PATCH	/api/v1/inspections/{id}/complete	INSP_UPDA TE	Baxışı tamamlama + WO yaratma
GET	/api/v1/inspections/due	INSP_READ	Baxış vaxtı çatmış texnikalar

POST /api/v1/inspections — Request (baxış başlatma):

```
{
  "vehicle_id": "uuid-of-vehicle",
  "checklist_id": "uuid-of-checklist",
  "inspector_id": "uuid-of-mechanic",
  "engine_hours_at_inspection": 1350.5,
  "inspection_date": "2026-06-29",
  "notes": ""
}
```

POST /api/v1/inspections/{id}/results — Maddə nəticələri (batch):

```
{
  "results": [
    {
      "checklist_item_id": "uuid-item-1", // Mühərrik yağı
      "selected_option": "Normal",
      "severity": "OK",
      "notes": ""
    },
    {
      "checklist_item_id": "uuid-item-2", // Yağ süzgəci
      "selected_option": "Dəyişdirmə lazım",
      "severity": "WARNING",
      "notes": "250 saatdır dəyişdirilməyib"
    },
    {
      "checklist_item_id": "uuid-item-3", // Cari motosaat
      "selected_option": "1350.5",
      "numeric_value": 1350.5,
      "severity": "OK"
    }
  ]
}
```

```
]
}
```

PATCH /api/v1/inspections/{id}/complete — Request:

```
{
  "overall_result": "MINOR_ISSUE",    // OK | MINOR_ISSUE | CRITICAL_ISSUE
  "notes":          "Yağ süzgəci növbəti baxışda dəyişdirilməlidir"
}
```

PATCH /api/v1/inspections/{id}/complete — Response (CRITICAL_ISSUE halı):

```
{
  "success": true,
  "data": {
    "inspection_id": "uuid",
    "status":        "COMPLETED",
    "overall_result": "CRITICAL_ISSUE",
    "vehicle_status": "IN_SERVICE",    // Avtomatik dəyişdirildi
    "generated_wo": {
      "wo_id": "uuid-of-new-wo",
      "wo_number": "WO-2026-0087",
      "priority": "CRITICAL",
      "status": "NEW"
    },
    "next_inspection": {
      "due_hours": 1600.5,    // +250 saat
      "due_date": "2026-07-29" // +30 gün
    }
  }
}
```

M18.3 Tamamlama Məntiqi — overall_result ⇒ Sistem Hərəkəti

overall_result	Sistem Hərəkəti	Detallar
OK	Heç nə	Texnika ACTIVE qalır. Növbəti baxış tarixi/saatı hesablanır. Motosaat M08-ə yazılır.
MINOR_ISSUE	WO yaradılır (Adi prioritet)	Texnika ACTIVE qalır. WO tipi: COMPANY. WO statusu: NEW. Servis Menecerinə bildiriş.
CRITICAL_ISSUE	WO yaradılır (Kritik prioritet)	Texnika statusu IN_SERVICE olur. WO tipi: COMPANY. WO statusu: NEW. Servis Meneceri + Admin-ə bildiriş.

M18.4 Biznes Qaydaları

Kod	Qayda
INS-V01	Yalnız COMPANY və INVESTOR qarajındakı texnikalar baxışa alına bilər → 422 + INSP_INVALID_VEHICLE_TYPE
INS-V02	Bir texnikanın eyni anda yalnız bir IN_PROGRESS baxışı ola bilər → 409 + INSP_ALREADY_IN_PROGRESS
INS-V03	CRITICAL_ISSUE seçildiyi halda overall_result göndərilmədən baxış tamamlana bilməz
INS-V04	engine_hours_at_inspection < vehicles.current_engine_hours → 422 + ENGINE_HOURS DECREASING
INS-V05	Baxış tamamlandıqdan sonra nəticələr dəyişdirilə bilməz (immutable) — yeni baxış lazımdır

Kod	Qayda
INS-V06	Checklist şablonunda is_default = true olan şablon avtomatik seçilir — admin dəyişdirə bilər
INS-V07	engine_hours_at_inspection M08-ə yazılır: entry_type = INSPECTION, source_ref_id = inspection.id
INS-V08	Növbəti baxış: trigger_type = BOTH olduqda hansı əvvəl çatarsa o aktiv sayılır

M19 — Anbar & Ehtiyat Hissələri (Warehouse)

Servis sisteminin daxilindəki anbar modulu. Layer məntiqli dinamik qovluq strukturu, stok idarəetməsi və Work Order-ə bağlı xərc hesablaması bu modulun əsasını təşkil edir.

M19.1 Verilənlər Bazı Sxemi

category_nodes — Dinamik qovluq strukturu (Recursive tree)

Layer məntiqli qovluq strukturu üçün özünə istinad edən (self-referencing) cədvəl istifadə olunur. Bu yanaşma istənilən dərinlikdə qovluq yaratmağa imkan verir.

```
CREATE TABLE ces_service.category_nodes (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id   UUID          NOT NULL REFERENCES branches(id),
  parent_id   UUID          NULL REFERENCES category_nodes(id), -- NULL = kök
  qovluq
  name        VARCHAR(255) NOT NULL,
  slug        VARCHAR(255) NOT NULL, -- "mueherrik-suzgecleri"
  depth       INTEGER       NOT NULL DEFAULT 0, -- 0=kök, 1=alt, 2=nəvə...
  path        TEXT          NOT NULL, -- "uuid1/uuid2/uuid3"
  sort_order  INTEGER       NOT NULL DEFAULT 0,
  is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  deleted_at  TIMESTAMPTZ   NULL,
  created_by  UUID          NOT NULL REFERENCES users(id),
  updated_by  UUID          NOT NULL REFERENCES users(id),
  UNIQUE (branch_id, parent_id, slug)
);

-- Bütün alt qovluqları tapmaq üçün recursive CTE nümunəsi:
WITH RECURSIVE subtree AS (
  SELECT * FROM category_nodes WHERE id = :rootId
  UNION ALL
  SELECT cn.* FROM category_nodes cn
  JOIN subtree s ON cn.parent_id = s.id
)
SELECT * FROM subtree WHERE deleted_at IS NULL;
```

spare_parts — Ehtiyat hissəsi / material kataloqu

```
CREATE TABLE ces_service.spare_parts (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id   UUID          NOT NULL REFERENCES branches(id),
  category_node_id UUID      NOT NULL REFERENCES category_nodes(id),
  name        VARCHAR(255) NOT NULL,
  sku         VARCHAR(100) NULL, -- Daxili kod
  unit        VARCHAR(50)  NOT NULL -- ədəd|litr|kq|metr|paket
  CHECK (unit IN ('PIECE', 'LITER', 'KG', 'METER', 'PACKAGE')),
  current_stock NUMERIC(10,3) NOT NULL DEFAULT 0,
  minimum_stock NUMERIC(10,3) NOT NULL DEFAULT 0,
  unit_cost     NUMERIC(15,2) NOT NULL DEFAULT 0, -- Son alış maya dəyəri
  default_margin_pct NUMERIC(5,2) NOT NULL DEFAULT 0,
  unit_sell_price NUMERIC(15,2) NOT NULL DEFAULT 0, -- Maya × (1+marja)
  supplier_name VARCHAR(255) NULL,
  last_purchased_at DATE      NULL,
  notes        TEXT          NULL,
  is_active    BOOLEAN       NOT NULL DEFAULT TRUE,
  created_at   TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  deleted_at   TIMESTAMPTZ   NULL,
  created_by   UUID          NOT NULL REFERENCES users(id),
  updated_by   UUID          NOT NULL REFERENCES users(id)
);
```

```

CREATE INDEX idx_spare_parts_branch_cat
  ON spare_parts(branch_id, category_node_id)
  WHERE deleted_at IS NULL AND is_active = TRUE;

-- Aşağı stok xəbərdarlığı üçün partial index:
CREATE INDEX idx_spare_parts_low_stock
  ON spare_parts(branch_id)
  WHERE current_stock <= minimum_stock AND deleted_at IS NULL;

```

stock_entries — Stok giriş qeydləri (Anbar qəbulu)

```

CREATE TABLE ces_service.stock_entries (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  spare_part_id     UUID                NOT NULL REFERENCES spare_parts(id),
  quantity          NUMERIC(10,3) NOT NULL,
  unit_cost         NUMERIC(15,2) NOT NULL, -- Bu alışın maya dəyəri
  total_cost        NUMERIC(15,2) NOT NULL, -- quantity × unit_cost
  supplier_name     VARCHAR(255) NULL,
  invoice_number    VARCHAR(100) NULL,
  entry_date        DATE                NOT NULL DEFAULT CURRENT_DATE,
  notes             TEXT                NULL,
  created_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  created_by        UUID                NOT NULL REFERENCES users(id),
  updated_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  updated_by        UUID                NOT NULL REFERENCES users(id)
);

```

spare_part_usages — Stok çıxışı (Work Order-ə bağlı)

```

CREATE TABLE ces_service.spare_part_usages (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  spare_part_id     UUID                NOT NULL REFERENCES spare_parts(id),
  wo_id             UUID                NOT NULL REFERENCES work_orders(id),
  wo_cost_item_id   UUID                NULL REFERENCES wo_cost_items(id),
  quantity          NUMERIC(10,3) NOT NULL,
  unit_cost         NUMERIC(15,2) NOT NULL, -- Çıxış anındakı maya dəyəri
  (qorunur)
  unit_sell_price   NUMERIC(15,2) NOT NULL, -- Çıxış anındakı satış qiyməti
  margin_pct        NUMERIC(5,2) NOT NULL,
  total_cost        NUMERIC(15,2) NOT NULL,
  total_sell        NUMERIC(15,2) NOT NULL,
  usage_date        DATE                NOT NULL DEFAULT CURRENT_DATE,
  notes             TEXT                NULL,
  created_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  created_by        UUID                NOT NULL REFERENCES users(id),
  updated_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  updated_by        UUID                NOT NULL REFERENCES users(id)
);

```

stock_adjustments — Stok düzəltmə qeydləri

```

CREATE TABLE ces_service.stock_adjustments (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  spare_part_id     UUID                NOT NULL REFERENCES spare_parts(id),
  quantity_before   NUMERIC(10,3) NOT NULL,
  quantity_after    NUMERIC(10,3) NOT NULL,
  delta             NUMERIC(10,3) NOT NULL, -- quantity_after - quantity_before
  reason            TEXT                NOT NULL, -- Məcburi: sayım fərqi, zay mal...
  adjustment_date   DATE                NOT NULL DEFAULT CURRENT_DATE,
  created_at        TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
  created_by        UUID                NOT NULL REFERENCES users(id),

```

```

    updated_at      TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    updated_by      UUID           NOT NULL REFERENCES users(id)
);

```

Stok dəyişikliyini avtomatik idarə edən trigger

```

-- stock_entries INSERT-dən sonra spare_parts.current_stock artır
CREATE OR REPLACE FUNCTION update_stock_on_entry()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE ces_service.spare_parts
    SET     current_stock      = current_stock + NEW.quantity,
           unit_cost          = NEW.unit_cost, -- Son alış qiyməti
           unit_sell_price    = NEW.unit_cost * (1 + default_margin_pct / 100),
           last_purchased_at = NEW.entry_date
    WHERE  id = NEW.spare_part_id;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_stock_entry
AFTER INSERT ON stock_entries
FOR EACH ROW EXECUTE FUNCTION update_stock_on_entry();

-- spare_part_usages INSERT-dən sonra stok azalır
CREATE OR REPLACE FUNCTION update_stock_on_usage()
RETURNS TRIGGER AS $$
BEGIN
    -- Stok yetərliliyini yoxla
    IF (SELECT current_stock FROM ces_service.spare_parts
        WHERE id = NEW.spare_part_id) < NEW.quantity THEN
        RAISE EXCEPTION 'STOCK_INSUFFICIENT';
    END IF;
    UPDATE ces_service.spare_parts
    SET     current_stock = current_stock - NEW.quantity
    WHERE  id = NEW.spare_part_id;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_stock_usage
BEFORE INSERT ON spare_part_usages
FOR EACH ROW EXECUTE FUNCTION update_stock_on_usage();

```

M19.2 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/warehouse/categories	WH_READ	Qovluq ağacı (recursive)
POST	/api/v1/warehouse/categories	WH_MANAGE	Yeni qovluq (parent_id ilə)
PUT	/api/v1/warehouse/categories/{id}	WH_MANAGE	Qovluq yeniləmə (ad, sıra)
DELETE	/api/v1/warehouse/categories/{id}	WH_MANAGE	Qovluq silmə (boş olmalıdır)
GET	/api/v1/warehouse/spare-parts	WH_READ	Hissə siyahısı (cat filtr, axtarış, stok)
GET	/api/v1/warehouse/spare-parts/{id}	WH_READ	Hissə detallı + stok tarixçəsi
POST	/api/v1/warehouse/spare-parts	WH_MANAGE	Yeni hissə / material əlavəsi

Metod	URL	İcazə	Açıqlama
PUT	/api/v1/warehouse/spare-parts/{id}	WH_MANAGE	Hissə məlumatları yeniləmə
PATCH	/api/v1/warehouse/spare-parts/{id}/margin	CHANGE_MARGIN	Default marja dəyişikliyi
POST	/api/v1/warehouse/spare-parts/{id}/stock-entry	WH_STOCK_IN	Stoka giriş (anbar qəbulu)
POST	/api/v1/warehouse/spare-parts/{id}/adjustment	WH_ADJUST	Stok düzəltmə (sayım fərqi)
GET	/api/v1/warehouse/spare-parts/{id}/movements	WH_READ	Stok hərəkət tarixçəsi
GET	/api/v1/warehouse/low-stock	WH_READ	Aşağı stok xəbərdarlıq siyahısı
POST	/api/v1/warehouse/usages	WH_USE	Work Order üçün hissə çıxışı
GET	/api/v1/warehouse/usages	WH_READ	Çıxış qeydləri (WO/hissə filtrlı)

POST /api/v1/warehouse/categories — Request:

```
{
  "parent_id": "uuid-of-parent-or-null", // null = kök qovluq
  "name": "Süzgəclər",
  "sort_order": 1
}
```

GET /api/v1/warehouse/categories — Response (ağac strukturu):

```
{
  "success": true,
  "data": [
    {
      "id": "uuid-1",
      "name": "Hissələr",
      "depth": 0,
      "children": [
        {
          "id": "uuid-2",
          "name": "Mühərrik",
          "depth": 1,
          "children": [
            { "id": "uuid-3", "name": "Süzgəclər", "depth": 2, "children": [] },
            { "id": "uuid-4", "name": "Kəmər & Zəncir", "depth": 2, "children": [] }
          ]
        }
      ]
    }
  ]
}
```

POST /api/v1/warehouse/spare-parts — Request:

```
{
  "category_node_id": "uuid-of-category",
  "name": "Yağ süzgəci Mann W940",
  "sku": "MANN-W940",
  "unit": "PIECE",
  "minimum_stock": 5,
  "unit_cost": 12.50,
}
```

```

"default_margin_pct": 25.00,
"supplier_name":      "AutoParts Azerbaijan",
"notes":              "CAT 320D, Komatsu PC200 üçün uyğundur"
}

```

POST /api/v1/warehouse/spare-parts/{id}/stock-entry — Request:

```

{
  "quantity":      20,
  "unit_cost":     11.80,          // Bu alışın maya dəyəri → spare_parts.unit_cost
  "supplier_name": "AutoParts Azerbaijan",
  "invoice_number": "INV-2026-0145",
  "entry_date":    "2026-06-29",
  "notes":         ""
}

```

POST /api/v1/warehouse/usages — Work Order üçün hissə çıxışı:

```

{
  "wo_id":          "uuid-of-work-order",
  "spare_part_id":  "uuid-of-spare-part",
  "quantity":       2,
  "margin_pct":     25.00,          // Default gəlir, dəyişdirilə bilər
  "notes":          "Yağ süzgəci dəyişimi üçün"
}

```

POST /api/v1/warehouse/usages — Response 201:

```

{
  "success": true,
  "data": {
    "id":          "uuid",
    "spare_part_id": "uuid",
    "spare_part_name": "Yağ süzgəci Mann W940",
    "wo_id":        "uuid",
    "quantity":     2,
    "unit_cost":    12.50,
    "unit_sell_price": 15.63,          // 12.50 × 1.25
    "margin_pct":   25.00,
    "total_cost":   25.00,
    "total_sell":   31.25,
    "stock_remaining": 18,          // Çıxışdan sonra qalan stok
    "low_stock_alert": false
  }
}

```

M19.3 Stok İdarəetmə Məntiqi

Stok dəyişiklikləri yalnız trigger vasitəsilə aparılır — Service katmanı birbaşa current_stock sahəsini UPDATE etmir. Bu, məlumat bütövlüyünü təmin edir.

Əməliyyat	Stok dəyişikliyi
Stok girişi (stock_entries INSERT)	current_stock += quantity (trigger)
WO üçün çıxış (spare_part_usages INSERT)	current_stock -= quantity (trigger) — əvvəl stok yoxlanılır

Əməliyyat	Stok dəyişikliyi
Stok düzəltmə (stock_adjustments INSERT)	current_stock = quantity_after (trigger)
Hissə silmə (soft delete)	current_stock dəyişmir — məlumat qorunur

M19.4 Biznes Qaydaları

Kod	Qayda
WH-V01	current_stock < usage.quantity → 422 + STOCK_INSUFFICIENT (DB trigger tərəfindən də qorunur)
WH-V02	Stok çıxışı yalnız aktiv (OPEN, IN_PROGRESS, ON_HOLD) Work Order-ə bağlı ola bilər → 422 + WO_NOT_ACTIVE
WH-V03	unit_sell_price < unit_cost → 422 + COST_BELOW_UNIT_COST (marja sıfıra endirilərsə xəbərdarlıq)
WH-V04	Marja dəyişikliyi yalnız CHANGE_MARGIN icazəsi → 403 + PERMISSION_DENIED, Audit Log-a yazılır
WH-V05	current_stock <= minimum_stock → Bildiriş göndərilir (M12) + low_stock_alert = true response-da
WH-V06	Qovluq silmə: içində aktiv hissə varsa → 422 + CATEGORY_NOT_EMPTY
WH-V07	Hissə silmə (soft delete): aktiv stok çıxışı olan hissə silinə bilməz → 422 + SPARE_PART_HAS_USAGES
WH-V08	spare_part_usages-dakı tarixi unit_cost dəyəri qorunur — sonrakı qiymət dəyişikliyi köhnə qeydlərə təsir etmir
WH-V09	stock_adjustments məcburi reason tələb edir — boş string qəbul edilmir → 400 + REASON_REQUIRED
WH-V10	quantity_after < 0 → 400 + NEGATIVE_STOCK_NOT_ALLOWED

M19.5 Stok Hesabatı SQL Nümunələri

Cari stok vəziyyəti (branch üzrə):

```
SELECT
    sp.id, sp.name, sp.sku, sp.unit,
    sp.current_stock,
    sp.minimum_stock,
    sp.unit_cost,
    sp.unit_sell_price,
    sp.current_stock * sp.unit_cost AS stock_value,
    CASE WHEN sp.current_stock <= sp.minimum_stock
        THEN TRUE ELSE FALSE END AS is_low_stock,
    cn.name AS category_name
FROM   ces_service.spare_parts sp
JOIN   ces_service.category_nodes cn ON sp.category_node_id = cn.id
WHERE  sp.branch_id = :branchId
    AND sp.deleted_at IS NULL
    AND sp.is_active = TRUE
ORDER BY cn.path, sp.name;
```

Work Order üzrə hissə istifadəsi:

```
SELECT
    wo.wo_number,
    sp.name AS spare_part,
    spu.quantity,
    spu.unit_cost,
    spu.unit_sell_price,
    spu.total_cost,
    spu.total_sell,
    spu.total_sell - spu.total_cost AS profit
FROM   ces_service.spare_part_usages spu
JOIN   ces_service.work_orders wo ON spu.wo_id = wo.id
JOIN   ces_service.spare_parts sp ON spu.spare_part_id = sp.id
WHERE  spu.branch_id = :branchId
      AND spu.usage_date BETWEEN :startDate AND :endDate
ORDER BY spu.usage_date DESC;
```

SOFTWARE REQUIREMENTS SPECIFICATION

CES Service Management System

Sənəd	SRS — Part 4: Completion & Output
Versiya	1.0
Tarix	29 İyun 2026
Hazırlayan	Sarkhan Babayev

M09 Arxiv • M10 Sənəd Generasiyası • M11 Hesabatlar • M12 Bildiriş • M13 ERP İntegrasiyası • M14 Mobil API

M09 — Arxiv Modulu

Tamamlanan (CLOSED statuslu) Work Order-lər avtomatik arxivə köçürülür. Arxiv immutable-dır — qeydlər silinə və dəyişdirilə bilməz. Yalnız oxuma, axtarış, export və sənəd çıxarma mümkündür.

M09.1 Arxivləmə Mexanizmi

Ayrıca arxiv cədvəli yoxdur. Work Order CLOSED statusuna keçdikdə is_archived = true olur. Arxiv sadəcə WO-nun filter görünüşüdür — məlumatlar eyni cədvəllərdə qalır.

work_orders cədvəlinə arxiv sütunları (migration)

```
-- V4__add_archive_fields_to_work_orders.sql
ALTER TABLE ces_service.work_orders
  ADD COLUMN is_archived      BOOLEAN      NOT NULL DEFAULT FALSE,
  ADD COLUMN archived_at      TIMESTAMPTZ  NULL,
  ADD COLUMN archived_by      UUID         NULL REFERENCES users(id),
  ADD COLUMN reopen_count     INTEGER      NOT NULL DEFAULT 0,
  ADD COLUMN last_reopened_at TIMESTAMPTZ  NULL,
  ADD COLUMN last_reopened_by UUID         NULL REFERENCES users(id);

-- Arxivdən sürətli axtarış üçün index:
CREATE INDEX idx_wo_archived ON work_orders(branch_id, is_archived, closed_at DESC)
  WHERE deleted_at IS NULL;
```

Avtomatik arxivləmə trigger-i

```
CREATE OR REPLACE FUNCTION auto_archive_on_close()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.status = 'CLOSED' AND OLD.status != 'CLOSED' THEN
    NEW.is_archived := TRUE;
    NEW.archived_at := NOW();
    NEW.archived_by := NEW.updated_by;
  END IF;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auto_archive
  BEFORE UPDATE ON work_orders
  FOR EACH ROW EXECUTE FUNCTION auto_archive_on_close();
```

M09.2 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/archive	ARCHIVE_READ	Arxivləşmiş WO-lar (filtr + paginasiya)
GET	/api/v1/archive/{id}	ARCHIVE_READ	Tam WO detalı (read-only)
GET	/api/v1/archive/{id}/cost-summary	ARCHIVE_READ	Maliyyə xülasəsi
GET	/api/v1/archive/{id}/documents	ARCHIVE_READ	Generasiya olunmuş sənədlər
POST	/api/v1/archive/{id}/reopen	REOPEN_ARCHIVE	Arxivdən yenidən aç (yeni WO yaranır)

Metod	URL	İcazə	Açıqlama
GET	/api/v1/archive/export	ARCHIVE_READ + EXPORT_REPORT	Excel/PDF export

GET /api/v1/archive — Query parametrləri:

Parametr	Tip	Açıqlama
customer_id	UUID	Müştəriyə görə filtr
vehicle_id	UUID	Texnikaya görə filtr
wo_type	string	PAID_SERVICE COMPANY INTERMEDIARY INSPECTION
date_from / date_to	date	Bağlanma tarixi aralığı
amount_min / amount_max	decimal	Ümumi məbləğ aralığı
search	string	WO nömrəsi, müştəri adı, texnika axtarışı (pg_trgm)
page, size, sort, dir	integer/string	Standart paginasiya

POST /api/v1/archive/{id}/reopen — Request:

```
{
  "reason": "Müştəri əlavə iş tələb etdi — yağ sızıntısı yenidən başladı"
}
```

POST /api/v1/archive/{id}/reopen — Response 201:

```
{
  "success": true,
  "data": {
    "original_wo_id": "uuid-of-original",
    "new_wo_id": "uuid-of-new-wo",
    "new_wo_number": "WO-2026-0099",
    "message": "Orijinal WO arxivdə qalır. Yeni WO NEW statusunda açıldı."
  }
}
```

M09.3 Biznes Qaydaları

Kod	Qayda
ARC-V01	Arxivdəki WO-nun heç bir sahəsi dəyişdirilə bilməz → 422 + ARCHIVE_IMMUTABLE
ARC-V02	Arxivdən yenidən açma orijinal WO-nu dəyişdirmir — yeni WO yaradılır, orijinal CLOSED qalır
ARC-V03	Yenidən açma yalnız REOPEN_ARCHIVE icazəsi ilə mümkündür
ARC-V04	Yenidən açma üçün reason məcburidir — Audit Log-a yazılır
ARC-V05	Export əməliyyatları (Excel/PDF) EXPORT_REPORT icazəsi tələb edir — Audit Log-a yazılır

M10 — Sənəd Generasiyası

Work Order məlumatları əsasında PDF sənədlər avtomatik generasiya olunur. Şablonlar dinamikdir — şirkət loqosu, rekvizitlər konfigurasiya oluna bilər.

M10.1 Verilənlər Bazı Sxemi

document_templates — Sənəd şablonları

```
CREATE TABLE ces_service.document_templates (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id   UUID          NOT NULL REFERENCES branches(id),
  doc_type    VARCHAR(50)   NOT NULL
              CHECK (doc_type IN ('SERVICE_ACT', 'INVOICE', 'OFFER',
                                  'HANDOVER_ACT', 'INTERMEDIARY_REPORT')),
  name        VARCHAR(255)  NOT NULL,
  is_default  BOOLEAN       NOT NULL DEFAULT FALSE,
  template_html TEXT        NOT NULL,    -- Thymeleaf şablonu (HTML → PDF)
  header_logo_url TEXT      NULL,
  company_name VARCHAR(255) NULL,
  company_voen VARCHAR(50)  NULL,
  company_address TEXT      NULL,
  company_phone VARCHAR(100) NULL,
  is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  created_by  UUID          NOT NULL REFERENCES users(id),
  updated_by  UUID          NOT NULL REFERENCES users(id)
);
```

generated_documents — Generasiya olunmuş sənədlər

```
CREATE TABLE ces_service.generated_documents (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id   UUID          NOT NULL REFERENCES branches(id),
  wo_id       UUID          NOT NULL REFERENCES work_orders(id),
  template_id  UUID          NOT NULL REFERENCES document_templates(id),
  doc_type    VARCHAR(50)   NOT NULL,
  doc_number  VARCHAR(100)  NOT NULL UNIQUE,    -- SRV-ACT-2026-0001
  file_name    VARCHAR(255) NOT NULL,
  file_url     TEXT         NOT NULL,
  file_size    BIGINT       NULL,
  generated_at TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  generated_by UUID          NOT NULL REFERENCES users(id),
  created_at   TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  created_by   UUID          NOT NULL REFERENCES users(id),
  updated_by   UUID          NOT NULL REFERENCES users(id)
);
```

M10.2 PDF Generasiya Texnologiyası

Spring Boot tərəfindən Thymeleaf + Flying Saucer (xhtmlrenderer) kombinasiyası ilə HTML → PDF çevirmə:

```
// PdfGeneratorService.java
@Service
public class PdfGeneratorService {

    @Autowired TemplateEngine templateEngine;

    public byte[] generate(String templateHtml, Map<String, Object> data) {
        Context ctx = new Context();
```



```

    ctx.setVariables(data);
    String html = templateEngine.process(templateHtml, ctx);

    ByteArrayOutputStream out = new ByteArrayOutputStream();
    ITextRenderer renderer = new ITextRenderer();
    renderer.setDocumentFromString(html);
    renderer.layout();
    renderer.createPDF(out);
    return out.toByteArray();
}

```

M10.3 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/document-templates	DOC_READ	Şablon siyahısı
POST	/api/v1/document-templates	DOC_MANAGE	Yeni şablon yaratma
PUT	/api/v1/document-templates/{id}	DOC_MANAGE	Şablon yeniləmə
DELETE	/api/v1/document-templates/{id}	DOC_MANAGE	Şablon silmə
POST	/api/v1/work-orders/{id}/documents	DOC_CREATE	WO üçün sənəd generasiya et
GET	/api/v1/work-orders/{id}/documents	DOC_READ	WO-nun sənəd siyahısı
GET	/api/v1/documents/{id}/download	DOC_READ	PDF faylını yüklə

POST /api/v1/work-orders/{id}/documents — Request:

```

{
  "doc_type": "SERVICE_ACT", //
  SERVICE_ACT|INVOICE|OFFER|HANDOVER_ACT|INTERMEDIARY_REPORT
  "template_id": "uuid-of-template" // null → default şablon istifadə olunur
}

```

Response 201:

```

{
  "success": true,
  "data": {
    "id": "uuid",
    "doc_type": "SERVICE_ACT",
    "doc_number": "SRV-ACT-2026-0023",
    "file_name": "servis_akti_WO-2026-0042.pdf",
    "download_url": "/api/v1/documents/uuid/download",
    "generated_at": "2026-06-29T14:30:00Z"
  }
}

```

M10.4 Thymeleaf Şablon Dəyişənləri

Hər şablon tipi üçün PDF-ə ötürülən data map-inin açarları:

Dəyişən	Mənası
<code>{company}</code>	Şirkət məlumatları (şablon konfigurasiyasından)

Dəyişkən	Mənası
<code>\${wo}</code>	Work Order əsas məlumatları (nömrə, tarix, status)
<code>\${vehicle}</code>	Texnika məlumatları (marka, model, şassi)
<code>\${customer}</code>	Müştəri məlumatları (ad, VÖEN, ünvan)
<code>\${costItems}</code>	Xərc maddələri siyahısı (description, qty, unit_cost, sell)
<code>\${services}</code>	Xidmət siyahısı (ad, tip, saat, qiymət)
<code>\${costSummary}</code>	Maliyyə xülasəsi (total_cost, total_sell, profit)
<code>\${generatedAt}</code>	Generasiya tarixi/vaxtı
<code>\${docNumber}</code>	Sənəd nömrəsi (SRV-ACT-2026-XXXX)

M11 — Hesabatlar & Analitika

Bütün hesabatlar dinamik filtr dəstəyi ilə PDF və Excel formatında export oluna bilər. Hesabat sorğuları read-only replika üzərindən aparılması tövsiyə olunur (yüksek yük zamanı).

M11.1 Hesabat Növləri & SQL Strukturu

1. Gəlir Hesabatı

```
SELECT
    DATE_TRUNC('month', wo.closed_at)          AS period,
    COUNT(wo.id)                               AS wo_count,
    SUM(wo.total_sell)                         AS total_revenue,
    SUM(wo.total_cost)                         AS total_cost,
    SUM(wo.total_sell - wo.total_cost)         AS total_profit,
    ROUND(AVG(wo.total_sell), 2)               AS avg_revenue_per_wo
FROM   ces_service.work_orders wo
WHERE  wo.branch_id = :branchId
      AND wo.status = 'CLOSED'
      AND wo.closed_at BETWEEN :startDate AND :endDate
      AND wo.deleted_at IS NULL
      AND (:woType IS NULL OR wo.wo_type = :woType)
GROUP BY DATE_TRUNC('month', wo.closed_at)
ORDER BY period;
```

2. Vasitəçilik Mənfəəti Hesabatı

```
SELECT
    c.name                               AS contractor_name,
    COUNT(sr.id)                         AS request_count,
    SUM(sr.customer_amount)              AS total_customer_amount,
    SUM(sr.contractor_amount)            AS total_contractor_amount,
    SUM(sr.intermediary_profit)           AS total_profit
FROM   ces_service.service_requests sr
JOIN   ces_service.contractors c ON sr.contractor_id = c.id
WHERE  sr.branch_id = :branchId
      AND sr.request_type = 'INTERMEDIARY'
      AND sr.status = 'CLOSED'
      AND sr.closed_at BETWEEN :startDate AND :endDate
      AND sr.deleted_at IS NULL
GROUP BY c.id, c.name
ORDER BY total_profit DESC;
```

3. Texnika Xərc Hesabatı

```
SELECT
    v.make || ' ' || v.model              AS vehicle_name,
    v.chassis_number,
    COUNT(wo.id)                         AS service_count,
    SUM(ci.total_cost)                   AS total_parts_cost,
    SUM(CASE WHEN ci.item_type = 'LABOR'
            THEN ci.total_cost ELSE 0 END) AS labor_cost,
    SUM(ci.total_sell_price)              AS total_sell
FROM   ces_service.vehicles v
JOIN   ces_service.work_orders wo ON wo.vehicle_id = v.id
JOIN   ces_service.wo_cost_items ci ON ci.wo_id = wo.id
WHERE  v.branch_id = :branchId
      AND wo.status = 'CLOSED'
      AND wo.closed_at BETWEEN :startDate AND :endDate
      AND v.deleted_at IS NULL
GROUP BY v.id, v.make, v.model, v.chassis_number
ORDER BY total_parts_cost DESC;
```

4. Mexanik İş Yükü Hesabatı

```
SELECT
    u.full_name                AS mechanic_name,
    COUNT(wo.id)               AS wo_count,
    COUNT(CASE WHEN wo.status = 'CLOSED'
            THEN 1 END)        AS completed_count,
    ROUND(AVG(EXTRACT(EPOCH FROM
                    (wo.closed_at - wo.created_at))/3600), 1)
                                AS avg_resolution_hours
FROM   ces_service.users u
JOIN   ces_service.work_orders wo ON wo.assigned_to = u.id
WHERE  wo.branch_id = :branchId
      AND wo.created_at BETWEEN :startDate AND :endDate
      AND wo.deleted_at IS NULL
GROUP BY u.id, u.full_name
ORDER BY completed_count DESC;
```

M11.2 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/reports/revenue	REPORT_R EAD	Gəlir hesabatı
GET	/api/v1/reports/cost	REPORT_R EAD	Xərc hesabatı
GET	/api/v1/reports/profit	REPORT_R EAD	Mənfəət & marja analizi
GET	/api/v1/reports/intermediary	REPORT_R EAD	Vasitəçilik hesabatı
GET	/api/v1/reports/vehicle	REPORT_R EAD	Texnika üzrə xərc hesabatı
GET	/api/v1/reports/mechanic-workload	REPORT_R EAD	Mexanik iş yükü
GET	/api/v1/reports/engine-hours	REPORT_R EAD	Motosaat dinamikası
GET	/api/v1/reports/spare-parts	REPORT_R EAD	Ehtiyat hissəsi istifadəsi
GET	/api/v1/reports/customer	REPORT_R EAD	Müştəri üzrə xidmət hesabatı
POST	/api/v1/reports/{type}/export	REPORT_R EAD + EXPORT_R EPORT	PDF/Excel export

GET /api/v1/reports/revenue — Query parametrləri:

Parametr	Tip	Açıqlama
date_from	date	Başlanğıc tarixi (məcburi)
date_to	date	Son tarix (məcburi)
group_by	string	day week month year (default: month)

Parametr	Tip	Açıqlama
wo_type	string	PAID_SERVICE COMPANY INTERMEDIARY (null = hamısı)
customer_id	UUID	Müştəri filtri (opsional)

POST /api/v1/reports/{type}/export — Request:

```
{
  "format": "EXCEL",          // PDF | EXCEL
  "date_from": "2026-01-01",
  "date_to": "2026-06-30",
  "group_by": "month"
}
```

Excel export: Apache POI (Spring Boot starter) ilə. PDF export: M10 ilə eyni Thymeleaf+iText pipeline-dan istifadə edir, hesabat şablonu ilə.

M12 — Bildiriş Mərkəzi

Sistem bildirişləri in-app kanalı üzərindən çatdırılır. Faza 2-də e-mail, Faza 3-də SMS/WhatsApp əlavə olunacaq.

M12.1 Verilənlər Bazı Sxemi

notifications cədvəli

```
CREATE TABLE ces_service.notifications (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NOT NULL REFERENCES branches(id),
  recipient_id      UUID                NOT NULL REFERENCES users(id),
  event_type        VARCHAR(100)        NOT NULL,
  title             VARCHAR(255)        NOT NULL,
  body              TEXT                 NOT NULL,
  ref_type          VARCHAR(50)         NULL,    -- WORK_ORDER | INSPECTION | VEHICLE ...
  ref_id            UUID                NULL,    -- İlgili entity UUID-si
  is_read           BOOLEAN              NOT NULL DEFAULT FALSE,
  read_at           TIMESTAMPTZ         NULL,
  created_at        TIMESTAMPTZ         NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ         NOT NULL DEFAULT NOW(),
  created_by        UUID                NOT NULL REFERENCES users(id),
  updated_by        UUID                NOT NULL REFERENCES users(id)
);

CREATE INDEX idx_notifications_recipient
ON notifications(recipient_id, is_read, created_at DESC);
```

M12.2 Hadisə Növləri (event_type)

event_type	Alıcı	Tetikləyici hadisə
WO_CREATED	Service Manager	Yeni Work Order yaradıldı
WO_ASSIGNED	Məsul mexanik	Work Order ona tapşırıldı
WO_STATUS_CHANGED	WO yaradıcısı + Manager	WO statusu dəyişdi
WO_COMPLETED	Service Manager	WO COMPLETED statusuna keçdi
WO_CLOSED_ACCOUNTING	Mühasib	WO bağlandı — mühasibatlığa push edildi
MARGIN_CHANGED	Admin + Service Manager	Marja dəyişdirildi
ENGINE_HOURS_THRESHOLD	Service Manager	Motosaat xəbərdarlıq həddinə çatdı
INSPECTION_DUE	Service Manager	Texniki baxış vaxtı yaxınlaşır
INSPECTION_CRITICAL	Service Manager + Admin	Texniki baxışda kritik problem tapıldı
LOW_STOCK	Anbardar + Service Manager	Ehtiyat hissəsi minimum həddə çatdı
STOCK_DEPLETED	Anbardar + Service Manager	Ehtiyat hissəsi sıfırlandı
ERP_SYNC_FAILED	Admin	CES ERP sinxronizasiyası uğursuz oldu

M12.3 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/notifications	öz bildirişləri	Bildiriş siyahısı (is_read filtri)
GET	/api/v1/notifications/unread-count	öz bildirişləri	Oxunmamış say
PATCH	/api/v1/notifications/{id}/read	öz bildirişləri	Bildirişi oxunmuş işarələ
PATCH	/api/v1/notifications/read-all	öz bildirişləri	Hamısını oxunmuş işarələ
DELETE	/api/v1/notifications/{id}	öz bildirişləri	Bildiriş silmə

GET /api/v1/notifications — Response:

```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "event_type": "WO_ASSIGNED",
      "title": "Yeni iş tapşırığı",
      "body": "WO-2026-0042 sizə tapşırıldı: Mühərrik yağı sızıntısı",
      "ref_type": "WORK_ORDER",
      "ref_id": "uuid-of-wo",
      "is_read": false,
      "created_at": "2026-06-29T10:15:00Z"
    }
  ],
  "meta": { "unread_count": 5, "total_items": 23 }
}
```

M12.4 Bildiriş Göndərmə Service

```
// NotificationService.java
@Service
public class NotificationService {

    public void send(NotificationEvent event) {
        // 1. Alıcıları tap (role/permission əsasında)
        List<UUID> recipients = resolveRecipients(
            event.getBranchId(), event.getEventType());

        // 2. Hər alıcı üçün DB-yə yaz
        recipients.forEach(userId -> {
            Notification n = Notification.builder()
                .branchId(event.getBranchId())
                .recipientId(userId)
                .eventType(event.getEventType())
                .title(event.getTitle())
                .body(event.getBody())
                .refType(event.getRefType())
                .refId(event.getRefId())
                .build();
            notificationRepo.save(n);
        });
    }
}
```

Real-time çatdırma: Faza 2-də Server-Sent Events (SSE) və ya WebSocket əlavə ediləcək. Hazırda polling (hər 30 saniyə GET /notifications/unread-count) kifayətdir.

M13 — CES ERP İnteqrasiyası

CES Service iki əsas nöqtədə CES ERP ilə inteqrasiya olunur: Müştəri bazası sinxronizasiyası (bidirectional) və Mühasibatlıq modulu məlumat ötürülməsi (unidirectional push).

M13.1 İnteqrasiya Arxitekturası

```
CES Service Backend
├── ErpApiClient (Spring @FeignClient)
│   ├── POST /api/customers → Müştəri yarat
│   ├── PUT /api/customers/{erpId} → Müştəri yenilə
│   ├── GET /api/customers → Müştəri siyahısı çək
│   └── POST /api/accounting/entries → Maliyyə qeydi göndər
└── RetryService (Redis-backed)
    └── Uğursuz ERP çağırışları → queue → max 3 retry, 5dəq fasilə
```

M13.2 Müştəri Bazası Sinxronizasiyası

erp_sync_logs — Sinxronizasiya tarixçəsi

```
CREATE TABLE ces_service.erp_sync_logs (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id   UUID          NOT NULL REFERENCES branches(id),
  sync_type    VARCHAR(50)  NOT NULL
                        CHECK (sync_type IN ('CUSTOMER_CREATE', 'CUSTOMER_UPDATE',
                        'CUSTOMER_PULL', 'ACCOUNTING_PUSH')),
  direction    VARCHAR(20)  NOT NULL CHECK (direction IN
('TO_ERP', 'FROM_ERP')),
  entity_id    UUID          NULL,      -- CES Service entity UUID
  erp_entity_id UUID        NULL,      -- ERP tərəfindəki UUID
  status       VARCHAR(20)  NOT NULL CHECK (status IN
('SUCCESS', 'FAILED', 'PENDING')),
  attempt_count INTEGER      NOT NULL DEFAULT 1,
  error_message TEXT         NULL,
  payload      JSONB         NULL,      -- Göndərilən/alınan data
  created_at   TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  created_by   UUID          NOT NULL REFERENCES users(id),
  updated_by   UUID          NOT NULL REFERENCES users(id)
);
```

ErpApiClient.java — Feign Client interfeysi

```
@FeignClient(name = "ces-erp", url = "${ces.erp.base-url}",
  configuration = ErpFeignConfig.class)
public interface ErpApiClient {

  @PostMapping("/api/customers")
  ErpCustomerResponse createCustomer(@RequestBody ErpCustomerRequest req);

  @PutMapping("/api/customers/{erpId}")
  ErpCustomerResponse updateCustomer(
    @PathVariable UUID erpId, @RequestBody ErpCustomerRequest req);

  @GetMapping("/api/customers")
  Page<ErpCustomerResponse> getCustomers(
    @RequestParam int page, @RequestParam int size);

  @PostMapping("/api/accounting/entries")
  void pushAccountingEntry(@RequestBody AccountingEntryRequest req);
```

}

M13.3 Mühasibatlıq Push — Payload Strukturu

Work Order CLOSED statusuna keçdikdə aşağıdakı payload CES ERP mühasibatlıq moduluna göndərilir:

```
{
  "entry_type": "SERVICE_INCOME",
  "wo_number": "WO-2026-0042",
  "branch_id": "uuid-of-branch",
  "customer_id": "erp-customer-uuid", // ERP-dəki UUID
  "closed_at": "2026-06-29T14:00:00Z",
  "wo_type": "PAID_SERVICE",
  "items": [
    {
      "type": "LABOR",
      "description": "Mexanik işçilik",
      "total_cost": 50.00,
      "total_sell": 65.00
    },
    {
      "type": "PART",
      "description": "Yağ süzgəci Mann W940 x2",
      "total_cost": 25.00,
      "total_sell": 31.25
    }
  ],
  "total_cost": 75.00,
  "total_sell": 96.25,
  "total_profit": 21.25
}
```

M13.4 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
POST	/api/v1/erp/customers/sync	SYNC_ERP	Müştəri bazasını ERP-dən çək
POST	/api/v1/erp/customers/{id}/push	SYNC_ERP	Müştərini ERP-yə push et
GET	/api/v1/erp/sync-logs	SYNC_ERP	Sinxronizasiya tarixçəsi
POST	/api/v1/erp/sync-logs/{id}/retry	SYNC_ERP	Uğursuz sinxronu yenidən cəhd et
GET	/api/v1/erp/sync-logs/failed	SYNC_ERP	Uğursuz sinxronizasiya siyahısı

M13.5 Xəta İdarəetməsi & Retry

- ERP çağırışı uğursuz olarsa → erp_sync_logs-a FAILED yazılır
- Redis queue-ya əlavə edilir: key = erp:retry:{syncLogId}, TTL = 24 saat
- Spring @Scheduled cron: hər 5 dəqiqədə bir PENDING/FAILED qeydlər yoxlanılır
- Max 3 cəhd — sonra FAILED olaraq qalır, Admin-ə bildiriş (ERP_SYNC_FAILED) göndərilir
- Admin manual retry edə bilər: POST /api/v1/erp/sync-logs/{id}/retry

```
// RetryScheduler.java
@Scheduled(fixedDelay = 300_000) // 5 dəqiqə
public void retryFailedSyncs() {
    List<ErpSyncLog> pending = syncLogRepo
```

```
.findByStatusAndAttemptCountLessThan('PENDING', 3);
pending.forEach(log -> {
    try {
        erpSyncService.retry(log);
    } catch (Exception e) {
        log.setAttemptCount(log.getAttemptCount() + 1);
        if (log.getAttemptCount() >= 3) {
            log.setStatus("FAILED");
            notificationService.send(ERP_SYNC_FAILED, log);
        }
        syncLogRepo.save(log);
    }
});
}
```

M14 — Mobil Tətbiq API (Faza 1)

İlkin mərhələdə mobil tətbiq yalnız motosaat qeydiyyatı funksionallığını əhatə edir. Bütün mövcud API endpoint-lərindən istifadə edir — ayrıca mobil endpoint yaradılmır. Mobil tətbiqin REST sorğuları eyni auth (JWT) və eyni URL strukturu ilə işləyir.

M14.1 Mobil üçün Xüsusi Fərqlər

Mövzu	Detal
Auth	Eyni JWT — mobil login üçün POST /api/v1/auth/login. refresh_token cihazda saxlanılır.
Branch header	Eyni X-Branch-Id header — mobil app token-dakı default branch-ı göndərir
entry_type	Motosaat qeydi zamanı "MOBILE" göndərilir — M08 API-si ilə eyni endpoint
Offline support	Faza 1-də offline dəstək yoxdur — şəbəkə tələb olunur
Platform	React Native (təvsiyə) və ya Flutter — bu SRS-in əhatə dairəsindən kənardır

M14.2 Mobil Tətbiqin İstifadə etdiyi Endpoint-lər

Metod	URL	İcazə	İstifadə
POST	/api/v1/auth/login	—	Mobil giriş
POST	/api/v1/auth/refresh	—	Token yenilə
POST	/api/v1/auth/logout	Bearer	Çıxış
GET	/api/v1/vehicles	VEHICLE_READ	Mexanikin texnikalar siyahısı
GET	/api/v1/vehicles/{id}/engine-hours/latest	EH_READ	Son motosaat dəyəri
POST	/api/v1/vehicles/{id}/engine-hours	EH_CREATE	Yeni motosaat qeydiyyatı (entry_type: MOBILE)
GET	/api/v1/notifications	öz	Bildirişlər
PATCH	/api/v1/notifications/{id}/read	öz	Oxunmuş işarəsi

M14.3 Mobil Axın Nümunəsi

Mexanik mobil tətbiqdən motosaat daxil edir:

1. POST /api/v1/auth/login
→ access_token + refresh_token + user.branches alınır
2. GET /api/v1/vehicles?garage_type=COMPANY
→ X-Branch-Id header göndərilir
→ Mexanikin şöbəsindəki texnikalar gəlir
3. GET /api/v1/vehicles/{id}/engine-hours/latest
→ Son motosaat dəyəri göstərilir (həm məlumat, həm validasiya üçün)
4. POST /api/v1/vehicles/{id}/engine-hours
Body: { "hours_value": 1350.5, "entry_type": "MOBILE",
"recorded_at": "2026-06-29", "notes": "" }

→ 201 yaradıldı; alerts_triggered yoxlanılır

5. Əgər alerts_triggered boş deyilsə → mobil ekranda xəbərdarlıq göstərilir

M14.4 Gələcək Mərhələlər (Faza 2+)

Faza	Funksionallıq	Əhatə
Faza 2	Work Order görüntüləmə	Mexanik öz tapşırıqlarını mobil tətbiqdən görür
Faza 2	Status yeniləmə	Mexanik WO statusunu mobil tətbiqdən dəyişir
Faza 2	Foto əlavəsi	WO-ya mobil kameradan foto yükləmə
Faza 3	Texniki baxış forması	Checklist mobil tətbiqdən doldurulur
Faza 3	Offline sync	Şəbəkəsiz işləmə + sinxronizasiya
Faza 3	Push notifications	FCM/APNs ilə real-time bildiriş
Faza 3	E-imza	Müştəri imzası mobil cihazda

SOFTWARE REQUIREMENTS SPECIFICATION

CES Service Management System

Sənəd	SRS — Part 5: Administration
Versiya	1.0
Tarix	29 İyun 2026
Hazırlayan	Sarkhan Babayev

M15 İstifadəçi İdarəetməsi • M16 Rol & İcazə (RBAC) • M17 Audit Log

M15 — İstifadəçi İdarəetməsi

Sistemdəki istifadəçilərin yaradılması, yenilənməsi, aktivləşdirilməsi/deaktivasiyası və şöbə + rol təyinatı bu modul üzərindən idarə olunur.

M15.1 Verilənlər Bazı Sxemi

users cədvəli

```
CREATE TABLE ces_service.users (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id   UUID          NOT NULL REFERENCES branches(id), -- əsas şöbə
  full_name    VARCHAR(255) NOT NULL,
  email       VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL, -- bcrypt
  phone       VARCHAR(50)   NULL,
  position     VARCHAR(100) NULL,      -- "Servis Meneceri", "Mexanik"...
  is_active    BOOLEAN      NOT NULL DEFAULT TRUE,
  last_login_at TIMESTAMPTZ NULL,
  failed_attempts INTEGER    NOT NULL DEFAULT 0,
  locked_until TIMESTAMPTZ  NULL,      -- brute-force kilidlənmə
  created_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
  deleted_at   TIMESTAMPTZ  NULL,
  created_by   UUID          NULL REFERENCES users(id), -- NULL = ilk Admin
  updated_by   UUID          NULL REFERENCES users(id)
);

CREATE INDEX idx_users_email ON users(email) WHERE deleted_at IS NULL;
CREATE INDEX idx_users_branch ON users(branch_id) WHERE deleted_at IS NULL;
```

user_branches — İstifadəçi çoxşöbə əlaqəsi

```
CREATE TABLE ces_service.user_branches (
  user_id      UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  branch_id    UUID NOT NULL REFERENCES branches(id) ON DELETE CASCADE,
  is_default    BOOLEAN NOT NULL DEFAULT FALSE,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (user_id, branch_id)
);
```

user_roles — İstifadəçi ↔ Rol əlaqəsi

```
CREATE TABLE ces_service.user_roles (
  user_id      UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role_id      UUID NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
  branch_id    UUID NOT NULL REFERENCES branches(id), -- Bu şöbə üçün rol
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  created_by   UUID NOT NULL REFERENCES users(id),
  PRIMARY KEY (user_id, role_id, branch_id)
);
```

user_roles.branch_id: İstifadəçi A-şöbəsində SERVICE_MANAGER, B-şöbəsində isə MECHANIC rolu daşıya bilər. Hər şöbə-rol kombinasiyası müstəqildir.

M15.2 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/users	USER_READ	Siyahı (branch, status, rol filtrlı)
GET	/api/v1/users/{id}	USER_READ	İstifadəçi detalı + rollar + şöbələr
POST	/api/v1/users	USER_CREATE	Yeni istifadəçi
PUT	/api/v1/users/{id}	USER_UPDATE	Məlumat yeniləmə
PATCH	/api/v1/users/{id}/activate	USER_UPDATE	Aktivləşdir
PATCH	/api/v1/users/{id}/deactivate	USER_UPDATE	Deaktiv et
DELETE	/api/v1/users/{id}	USER_DELETE	Soft delete
POST	/api/v1/users/{id}/roles	USER_ROLE_MANAGE	Rol təyinatı
DELETE	/api/v1/users/{id}/roles/{roleId}	USER_ROLE_MANAGE	Rol ləğvi
POST	/api/v1/users/{id}/branches	USER_UPDATE	Şöbə əlavəsi
DELETE	/api/v1/users/{id}/branches/{branchId}	USER_UPDATE	Şöbə çıxarılması
PATCH	/api/v1/users/{id}/reset-password	USER_UPDATE	Admin tərəfindən şifrə sıfırlama
GET	/api/v1/users/{id}/activity	USER_READ	Son giriş + fəaliyyət xülasəsi

POST /api/v1/users — Request:

```
{
  "full_name":    "Əli Hüseynov",
  "email":        "ali@ces.az",
  "password":     "Temp@123",           // İlk giriş zamanı dəyişdirilməli
  "phone":        "+994501234567",
  "position":     "Mexanik",
  "branch_id":    "uuid-of-primary-branch",
  "role_ids": [
    { "role_id": "uuid-of-mechanic-role", "branch_id": "uuid-of-branch" }
  ]
}
```

POST /api/v1/users/{id}/roles — Request:

```
{
  "role_id":    "uuid-of-role",
  "branch_id":  "uuid-of-branch"
}
```

M15.3 Biznes Qaydaları

Kod	Qayda
USR-V01	Email unikal olmalıdır (bütün sistem üzrə) → 409 + DUPLICATE_EMAIL
USR-V02	Şifrə: min 8 simvol, ən azı 1 böyük hərf, 1 rəqəm → 400 + WEAK_PASSWORD
USR-V03	Öz hesabını deaktiv edə bilməz → 422 + CANNOT_DEACTIVATE_SELF
USR-V04	Aktiv Work Order-i olan istifadəçi silinə bilməz → 422 + USER_HAS_ACTIVE_WO
USR-V05	Silindikdə istifadəçinin yaratdığı bütün qeydlər qorunur — tarixi pozulmur
USR-V06	Rol təyinatında rol.branch_id istifadəçinin user_branches-ında olmalıdır
USR-V07	is_active = false olan istifadəçi sistemə giriş edə bilməz → 401 + AUTH_ACCOUNT_INACTIVE
USR-V08	Admin öz Admin rolunu ləğv edə bilməz (sistemdə ən az 1 Admin olmalıdır) → 422 + LAST_ADMIN

M16 — Rol & İcazə İdarəetməsi (RBAC)

Sistemin dinamik RBAC (Role-Based Access Control) modulu. Administrator istənilən vaxt yeni rol yarada, mövcud rol dəyişdirə, hər modul üzrə GET/POST/PUT/DELETE və xüsusi biznes icazələrini konfigurasiya edə bilər. Button-level icazələr daxil.

M16.1 Verilənlər Bazı Sxemi

roles cədvəli

```
CREATE TABLE ces_service.roles (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id   UUID          NOT NULL REFERENCES branches(id),
  name        VARCHAR(100)  NOT NULL,    -- "Servis Meneceri", "Mexanik"
  code        VARCHAR(100)  NOT NULL,    -- "SERVICE_MANAGER", "MECHANIC"
  description TEXT          NULL,
  is_system   BOOLEAN       NOT NULL DEFAULT FALSE, -- sistem rolu silinə bilməz
  is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  deleted_at  TIMESTAMPTZ   NULL,
  created_by  UUID          NOT NULL REFERENCES users(id),
  updated_by  UUID          NOT NULL REFERENCES users(id),
  UNIQUE (branch_id, code)
);
```

permissions cədvəli — İcazə kataloqu

```
CREATE TABLE ces_service.permissions (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  code        VARCHAR(100)  NOT NULL UNIQUE, -- "WO_CREATE", "CHANGE_MARGIN"
  name        VARCHAR(255)  NOT NULL,        -- "Work Order Yarat"
  description TEXT          NULL,
  module      VARCHAR(100)  NOT NULL,        -- "WORK_ORDER", "COST"
  perm_type   VARCHAR(50)   NOT NULL
  CHECK (perm_type IN ('CRUD', 'BUSINESS', 'REPORT', 'SYSTEM')),
  http_method VARCHAR(20)   NULL,            -- GET|POST|PUT|DELETE (CRUD üçün)
  is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  created_by  UUID          NOT NULL REFERENCES users(id),
  updated_by  UUID          NOT NULL REFERENCES users(id)
);
```

role_permissions — Rol ↔ İcazə əlaqəsi

```
CREATE TABLE ces_service.role_permissions (
  role_id       UUID NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
  permission_id UUID NOT NULL REFERENCES permissions(id) ON DELETE CASCADE,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  created_by    UUID NOT NULL REFERENCES users(id),
  PRIMARY KEY (role_id, permission_id)
);
```

M16.2 İcazə Kataloqu — Tam Siyahı

Bu icazələr sistem başlatıldığında Flyway migration ilə permissions cədvəlinə yazılır. Admin sonradan yeni icazə əlavə edə bilər.

VEHICLE (Qaraj) İcazələri

Kod	Modul	Tip	Açıqlama
VEHICLE_READ	VEHICLE	CRUD (GET)	Texnika siyahısı + detallı
VEHICLE_CREATE	VEHICLE	CRUD (POST)	Yeni texnika
VEHICLE_UPDATE	VEHICLE	CRUD (PUT)	Texnika yeniləmə + status dəyişikliyi
VEHICLE_DELETE	VEHICLE	CRUD (DELETE)	Texnika soft delete
VEHICLE_OVERRIDE	VEHICLE	BUSINESS	IN_SERVICE texnikaya override ilə yeni sorğu

CUSTOMER (Müştəri) İcazələri

Kod	Modul	Tip	Açıqlama
CUSTOMER_READ	CUSTOMER	CRUD (GET)	Müştəri siyahısı + detallı
CUSTOMER_CREATE	CUSTOMER	CRUD (POST)	Yeni müştəri + ERP sinxron
CUSTOMER_UPDATE	CUSTOMER	CRUD (PUT)	Müştəri yeniləmə + ERP sinxron
CUSTOMER_DELETE	CUSTOMER	CRUD (DELETE)	Müştəri soft delete
CUSTOMER_SYNC	CUSTOMER	BUSINESS	ERP-dən manual sinxronizasiya başlat

SERVICE REQUEST (Servis Sorğusu) İcazələri

Kod	Modul	Tip	Açıqlama
SR_READ	SERVICE_REQUEST	CRUD (GET)	Sorğu siyahısı + detallı
SR_CREATE	SERVICE_REQUEST	CRUD (POST)	Yeni sorğu yaratma
SR_UPDATE	SERVICE_REQUEST	CRUD (PUT)	Sorğu yeniləmə + tapşırıq əlavəsi
SR_DELETE	SERVICE_REQUEST	CRUD (DELETE)	Sorğu ləğv etmə (yalnız OPEN)
CONTRACTOR_READ	SERVICE_REQUEST	CRUD (GET)	Podratçılar siyahısı
CONTRACTOR_CREATE	SERVICE_REQUEST	CRUD (POST)	Yeni podratçı

WORK ORDER İcazələri

Kod	Modul	Tip	Açıqlama
WO_READ	WORK_ORDER	CRUD (GET)	WO siyahısı + detalı + tarixçə
WO_CREATE	WORK_ORDER	CRUD (POST)	Manuel WO yaratma
WO_UPDATE	WORK_ORDER	CRUD (PUT)	Status dəyişikliyi, şərh, fayl
WO_DELETE	WORK_ORDER	CRUD (DELETE)	WO soft delete
WO_CLOSE	WORK_ORDER	BUSINESS	WO-nu CLOSED statusuna keçirmək
WO_CANCEL	WORK_ORDER	BUSINESS	WO-nu CANCELLED statusuna keçirmək
APPROVE_COST	WORK_ORDER	BUSINESS	WO xərclərini təsdiqləmək

COST & MARGIN İcazələri

Kod	Modul	Tip	Açıqlama
COST_READ	COST	CRUD (GET)	Xərc maddələri + xülasə
COST_MANAGE	COST	CRUD (POST/PUT/DELETE)	Xərc maddəsi əlavə/dəyişiklik/silmə
CHANGE_MARGIN	COST	BUSINESS	Marja dəyişikliyi (audit log yazılır)
VIEW_COST_DETAILS	COST	BUSINESS	Daxili maya dəyəri görmək (mexanikdən gizlənə bilər)
MARGIN_CONFIG_READ	COST	CRUD (GET)	Sistem default marjalarını görmək
MARGIN_CONFIG_UPDATE	COST	BUSINESS	Sistem default marjalarını dəyişmək

ENGINE HOURS (Motosaat) İcazələri

Kod	Modul	Tip	Açıqlama
EH_READ	ENGINE_HOURS	CRUD (GET)	Motosaat tarixçəsi + son dəyər
EH_CREATE	ENGINE_HOURS	CRUD (POST)	Yeni motosaat qeydiyyatı
EH_ALERT_MANAGE	ENGINE_HOURS	BUSINESS	Motosaat xəbərdarlıqlarını idarə et

INSPECTION (Texniki Baxış) İcazələri

Kod	Modul	Tip	Açıqlama
INSP_READ	INSPECTION	CRUD (GET)	Baxış siyahısı + detallı + planlar
INSP_CREATE	INSPECTION	CRUD (POST)	Yeni baxış başlatma
INSP_UPDATE	INSPECTION	CRUD (PUT)	Nəticə daxil etmə + tamamlama
INSP_MANAGE	INSPECTION	BUSINESS	Checklist şablonu + baxış planı idarəsi

WAREHOUSE (Anbar) İcazələri

Kod	Modul	Tip	Açıqlama
WH_READ	WAREHOUSE	CRUD (GET)	Hissə + qovluq siyahısı, stok
WH_MANAGE	WAREHOUSE	CRUD (POST/PUT/DELETE)	Hissə + qovluq yaratma/yeniləmə/silmə
WH_STOCK_IN	WAREHOUSE	BUSINESS	Stoka giriş (anbar qəbulu)
WH_USE	WAREHOUSE	BUSINESS	WO üçün stok çıxışı
WH_ADJUST	WAREHOUSE	BUSINESS	Stok düzəltmə (sayım fərqi)

ARCHIVE, DOCUMENT, REPORT İcazələri

Kod	Modul	Tip	Açıqlama
ARCHIVE_READ	ARCHIVE	CRUD (GET)	Arxiv siyahısı + detallı
REOPEN_ARCHIVE	ARCHIVE	BUSINESS	Arxivdən yenidən aç
DOC_READ	DOCUMENT	CRUD (GET)	Sənəd siyahısı + yükləmə
DOC_CREATE	DOCUMENT	CRUD (POST)	Sənəd generasiya et
DOC_MANAGE	DOCUMENT	BUSINESS	Sənəd şablonu idarəsi
REPORT_READ	REPORT	CRUD (GET)	Hesabatları görüntülə
EXPORT_REPORT	REPORT	BUSINESS	Hesabatları PDF/Excel-ə ixrac et

ERP, BRANCH, USER, ROLE, AUDIT İcazələri

Kod	Modul	Tip	Açıqlama
SYNC_ERP	ERP	BUSINESS	ERP sinxronizasiyasını idarə et
BRANCH_VIEW_OW N	BRANCH	BUSINESS	Yalnız öz şöbəsinə görünür (default)
BRANCH_VIEW_ALL	BRANCH	BUSINESS	Bütün şöbələri görünür (CEO, Baş Mühasib)
BRANCH_MANAGE	BRANCH	BUSINESS	Şöbə yaratma / yeniləmə
USER_READ	USER	CRUD (GET)	İstifadəçi siyahısı + detallı
USER_CREATE	USER	CRUD (POST)	Yeni istifadəçi yaratma
USER_UPDATE	USER	CRUD (PUT)	İstifadəçi yeniləmə + şifrə sıfırlama
USER_DELETE	USER	CRUD (DELETE)	İstifadəçi soft delete
USER_ROLE_MANA GE	USER	BUSINESS	İstifadəçiyə rol təyin/ləğv et
ROLE_READ	ROLE	CRUD (GET)	Rol siyahısı + icazə matrisi
ROLE_CREATE	ROLE	CRUD (POST)	Yeni rol yaratma
ROLE_UPDATE	ROLE	CRUD (PUT)	Rol yeniləmə + icazə dəyişikliyi
ROLE_DELETE	ROLE	CRUD (DELETE)	Rol silmə (is_system = false)
AUDIT_READ	AUDIT	CRUD (GET)	Audit log görüntüləmə + export

M16.3 Default Rol ↔ İcazə Matrisi

Aşağıdakı matris başlanğıc default konfigurasiyadır. Admin istənilən icazəni istənilən rola əlavə edə / çıxara bilər.

Format: ✓ = icazəli, – = qadağan, ≈ = məhdud (yalnız öz şöbəsi / öz tapşırıqları)

İcazə Kodu	Admin	ServisMə neceri	Mexanik	Mühasib	Anbarda r	Direktor
VEHICLE_READ	✓	✓	✓	–	–	✓
VEHICLE_CREATE	✓	✓	–	–	–	–
VEHICLE_UPDATE	✓	✓	–	–	–	–
VEHICLE_DELETE	✓	–	–	–	–	–
VEHICLE_OVERRIDE	✓	✓	–	–	–	–
CUSTOMER_READ	✓	✓	≈	✓	–	✓
CUSTOMER_CREATE	✓	✓	–	–	–	–

İcazə Kodu	Admin	ServisMeceri	Mexanik	Mühasib	Anbardar	Direktor
CUSTOMER_UPDATE	✓	✓	—	—	—	—
CUSTOMER_SYNC	✓	—	—	—	—	—
SR_READ	✓	✓	≈	✓	—	✓
SR_CREATE	✓	✓	—	—	—	—
SR_UPDATE	✓	✓	≈	—	—	—
WO_READ	✓	✓	≈	✓	—	✓
WO_UPDATE	✓	✓	≈	—	—	—
WO_CLOSE	✓	✓	—	—	—	—
WO_CANCEL	✓	✓	—	—	—	—
APPROVE_COST	✓	✓	—	✓	—	—
COST_READ	✓	✓	—	✓	—	—
COST_MANAGE	✓	✓	—	—	—	—
CHANGE_MARGIN	✓	✓	—	—	—	—
VIEW_COST_DETAILS	✓	✓	—	✓	—	—
MARGIN_CONFIG_UPDATE	✓	—	—	—	—	—
EH_READ	✓	✓	✓	—	—	✓
EH_CREATE	✓	✓	✓	—	—	—
EH_ALERT_MANAGE	✓	✓	—	—	—	—
INSP_READ	✓	✓	✓	—	—	✓
INSP_CREATE	✓	✓	✓	—	—	—
INSP_UPDATE	✓	✓	✓	—	—	—
INSP_MANAGE	✓	✓	—	—	—	—
WH_READ	✓	✓	✓	✓	✓	✓
WH_MANAGE	✓	✓	—	—	✓	—
WH_STOCK_IN	✓	✓	—	—	✓	—
WH_USE	✓	✓	✓	—	—	—
WH_ADJUST	✓	✓	—	—	✓	—
ARCHIVE_READ	✓	✓	—	✓	—	✓
REOPEN_ARCHIVE	✓	✓	—	—	—	—
DOC_CREATE	✓	✓	—	✓	—	—
REPORT_READ	✓	✓	—	✓	—	✓
EXPORT_REPORT	✓	✓	—	✓	—	—
SYNC_ERP	✓	—	—	—	—	—
BRANCH_VIEW_ALL	✓	—	—	—	—	✓
BRANCH_MANAGE	✓	—	—	—	—	—
USER_READ	✓	✓	—	—	—	—
USER_CREATE	✓	—	—	—	—	—
USER_UPDATE	✓	—	—	—	—	—

İcazə Kodu	Admin	ServisMeceri	Mexanik	Mühasib	Anbardar	Direktor
USER_ROLE_MANAGE	✓	—	—	—	—	—
ROLE_READ	✓	✓	—	—	—	—
ROLE_CREATE	✓	—	—	—	—	—
ROLE_UPDATE	✓	—	—	—	—	—
ROLE_DELETE	✓	—	—	—	—	—
AUDIT_READ	✓	—	—	—	—	—

M16.4 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/roles	ROLE_READ	Rol siyahısı
GET	/api/v1/roles/{id}	ROLE_READ	Rol detallı + icazələr
POST	/api/v1/roles	ROLE_CREATE	Yeni rol
PUT	/api/v1/roles/{id}	ROLE_UPDATE	Rol ad/təsvir yeniləmə
DELETE	/api/v1/roles/{id}	ROLE_DELETE	Rol silmə (is_system=false)
GET	/api/v1/roles/{id}/permissions	ROLE_READ	Rolün icazə siyahısı
POST	/api/v1/roles/{id}/permissions	ROLE_UPDATE	Rola icazə əlavəsi
DELETE	/api/v1/roles/{id}/permissions/{pId}	ROLE_UPDATE	Roldan icazə çıxarma
GET	/api/v1/permissions	ROLE_READ	Bütün icazə kataloqu
GET	/api/v1/permissions/matrix	ROLE_READ	Tam Rol × İcazə matrisi

POST /api/v1/roles — Request:

```
{
  "name": "Sahə Mexaniki",
  "code": "FIELD_MECHANIC",
  "description": "Yalnız motosaat qeydiyyatı və mobil baxış",
  "permission_ids": [
    "uuid-of-EH_READ", "uuid-of-EH_CREATE",
    "uuid-of-VEHICLE_READ", "uuid-of-WO_READ"
  ]
}
```

M16.5 JWT-də İcazə Yüklənməsi

İstifadəçi login olduqda bütün rollarının icazələri birləşdirilir (union) və JWT-nin permissions sahəsinə yazılır:

```
// AuthService.java
List<String> permissions = userRoleRepo
    .findByUserIdAndBranchId(userId, branchId)
    .stream()
    .flatMap(ur -> ur.getRole().getPermissions().stream())
    .map(Permission::getCode)
    .distinct()
    .collect(Collectors.toList());

// JWT payload-a yazılır:
claims.put("permissions", permissions);
// ["WO_READ", "WO_UPDATE", "EH_CREATE", "CHANGE_MARGIN", ...]
```

Cache: İcazə siyahısı Redis-də saxlanılır (key: perm:{userId}:{branchId}, TTL: 15 dəq). Rol dəyişikliyi olduqda cache invalidasiya edilir.

M16.6 Biznes Qaydaları

Kod	Qayda
RBAC-V01	is_system = true olan rol silinə bilməz → 422 + SYSTEM_ROLE_PROTECTED
RBAC-V02	Rol kodu (code) branch daxilində unikal olmalıdır → 409 + DUPLICATE_ROLE_CODE
RBAC-V03	Aktiv istifadəçisi olan rol silinə bilməz → 422 + ROLE_HAS_ACTIVE_USERS
RBAC-V04	Bütün icazə dəyişiklikləri Audit Log-a yazılır (əlavə olunan/çıxarılan icazə kodu)
RBAC-V05	Branch dəyişikliyi zamanı (switch-branch) yeni branch üçün icazələr yenidən yüklənir
RBAC-V06	icazə kataloqu (permissions cədvəli) yalnız migration ilə dəyişdirilir — admin UI-dən dəyişdirilmir

M17 — Audit Log

Bütün kritik əməliyyatlar dəyişdirilməz (immutable) şəkildə Audit Log-da qeydə alınır. Audit Log yalnız Admin və Auditor rolları tərəfindən oxuna bilər; heç bir istifadəçi tərəfindən silinə bilməz.

M17.1 Verilənlər Bazı Sxemi

audit_logs cədvəli

```
CREATE TABLE ces_service.audit_logs (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id         UUID                NULL,          -- NULL = sistem səviyyəsi hadisə
  user_id           UUID                NULL,          -- NULL = sistem (scheduled job)
  user_email        VARCHAR(255)       NULL,          -- Snapshot: silinsə də qalır
  user_full_name    VARCHAR(255)       NULL,
  ip_address        INET                NULL,
  user_agent        TEXT                NULL,
  event_type        VARCHAR(100)       NOT NULL,
  module            VARCHAR(100)       NOT NULL,
  action            VARCHAR(100)       NOT NULL,      --
CREATE | READ | UPDATE | DELETE | BUSINESS
  entity_type       VARCHAR(100)       NULL,          -- WORK_ORDER | VEHICLE | USER...
  entity_id         UUID                NULL,
  entity_number     VARCHAR(100)       NULL,          -- WO-2026-0042 (oxumaq üçün)
  old_value         JSONB              NULL,          -- Dəyişiklikdən öncəki vəziyyət
  new_value         JSONB              NULL,          -- Dəyişiklikdən sonrakı vəziyyət
  result            VARCHAR(20)       NOT NULL        -- SUCCESS | FAILED | DENIED
              CHECK (result IN ('SUCCESS', 'FAILED', 'DENIED')),
  error_message     TEXT                NULL,
  created_at        TIMESTAMPTZ       NOT NULL DEFAULT NOW()
  -- NOT NULL created_by burada yoxdur – sistem özü yazır
);

-- Audit log heç vaxt silinmir – DELETE icazəsi yoxdur
-- Əlavə təminat: RLS ilə DELETE bloku (opsional)
CREATE INDEX idx_audit_branch_time ON audit_logs(branch_id, created_at DESC);
CREATE INDEX idx_audit_user_time   ON audit_logs(user_id, created_at DESC);
CREATE INDEX idx_audit_entity     ON audit_logs(entity_type, entity_id);
CREATE INDEX idx_audit_event_type ON audit_logs(event_type, created_at DESC);
```

M17.2 Audit Hadisə Kataloqu

event_type	Modul	Qeydə alınan məlumat
USER_LOGIN	AUTH	user_id, IP, user_agent, result (SUCCESS/FAILED)
USER_LOGOUT	AUTH	user_id, session müddəti
USER_LOGIN_FAILED	AUTH	email, IP, cəhd sayı
USER_ACCOUNT_LOCKED	AUTH	user_id, IP, lock müddəti
PASSWORD_RESET	AUTH	user_id, kim tərəfindən (admin/self)
TOKEN_REVOKED	AUTH	user_id, jti
WO_STATUS_CHANGED	WORK_ORDER	entity_id, old_value.status, new_value.status, notes
WO_CLOSED	WORK_ORDER	entity_id, total_cost, total_sell, closed_by

event_type	Modul	Qeydə alınan məlumat
WO_CANCELLED	WORK_ORDER	entity_id, reason
MARGIN_CHANGED	COST	entity_id (wo_cost_item), old_value.margin_pct, new_value.margin_pct
COST_ITEM_ADDED	COST	wo_id, item_type, description, unit_cost, margin_pct, total_sell
STOCK_ENTRY	WAREHOUSE	spare_part_id, quantity, unit_cost, previous_stock, new_stock
STOCK_USAGE	WAREHOUSE	spare_part_id, wo_id, quantity, previous_stock, new_stock
STOCK_ADJUSTMENT	WAREHOUSE	spare_part_id, quantity_before, quantity_after, reason
INSPECTION_COMPLETE	INSPECTION	entity_id, overall_result, generated_wo_id
DOCUMENT_GENERATED	DOCUMENT	wo_id, doc_type, doc_number, generated_by
REPORT_EXPORTED	REPORT	report_type, format, date_range, exported_by
ROLE_PERMISSION_CHANGED	ROLE	role_id, added_permissions[], removed_permissions[]
USER_ROLE_ASSIGNED	USER	user_id, role_id, branch_id, assigned_by
USER_ROLE_REVOKED	USER	user_id, role_id, branch_id, revoked_by
ERP_SYNC_SUCCESS	ERP	sync_type, entity_id, erp_entity_id
ERP_SYNC_FAILED	ERP	sync_type, entity_id, error_message, attempt_count
ARCHIVE_REOPENED	ARCHIVE	original_wo_id, new_wo_id, reason, reopened_by

M17.3 Audit Log Yazma Mexanizmi

Spring AOP (Aspect-Oriented Programming) ilə bütün kritik service metodları avtomatik loqlanır:

```
// AuditAspect.java
@Aspect
@Component
public class AuditAspect {

    @Around("@annotation(Auditable)")
    public Object audit(ProceedingJoinPoint pjp,
        Auditable auditable) throws Throwable {
        AuditLog log = AuditLog.builder()
            .branchId(BranchContext.get())
            .userId(SecurityContext.getCurrentUserId())
            .userEmail(SecurityContext.getCurrentUserEmail())
            .ipAddress(RequestContext.getIp())
            .eventType(auditable.eventType())
            .module(auditable.module())
            .action(auditable.action())
            .build();

        try {
            Object result = pjp.proceed();
            log.setResult("SUCCESS");
            return result;
        }
    }
}
```

```

    } catch (Exception e) {
        log.setResult("FAILED");
        log.setErrorMessage(e.getMessage());
        throw e;
    } finally {
        auditLogRepo.save(log);    // async - performansə təsir etməsin
    }
}

// istifadə nümunəsi:
@Auditable(eventType="MARGIN_CHANGED", module="COST", action="BUSINESS")
@PreAuthorize("hasAuthority('CHANGE_MARGIN')")
public WoCostItemResponse updateMargin(UUID itemId, BigDecimal newMargin) {
    // ...
}

```

M17.4 API Endpoint-ləri

Metod	URL	İcazə	Açıqlama
GET	/api/v1/audit-logs	AUDIT_READ	Audit log siyahısı (filtr + paginasiya)
GET	/api/v1/audit-logs/{id}	AUDIT_READ	Tək qeyd detallı (old/new value)
POST	/api/v1/audit-logs/export	AUDIT_READ	Excel export
GET	/api/v1/audit-logs/entity/{type}/{id}	AUDIT_READ	Entity üzrə bütün tarixçə
GET	/api/v1/audit-logs/user/{userId}	AUDIT_READ	İstifadəçi fəaliyyət tarixçəsi

GET /api/v1/audit-logs — Query parametrləri:

Parametr	Tip	Açıqlama
event_type	string	Hadisə tipi filtri (USER_LOGIN, MARGIN_CHANGED...)
module	string	Modul filtri (AUTH, COST, WORK_ORDER...)
user_id	UUID	İstifadəçiyə görə filtr
entity_type	string	Entity tipi (WORK_ORDER, VEHICLE...)
entity_id	UUID	Konkret entity üzrə filtr
result	string	SUCCESS FAILED DENIED
date_from / date_to	datetime	Tarix-vaxt aralığı
search	string	entity_number, user_email üzrə axtarış
page, size, sort, dir	—	Standart paginasiya (default: created_at DESC)

GET /api/v1/audit-logs/{id} — Response:

```

{
  "success": true,
  "data": {
    "id": "uuid",
    "user_email": "manager@ces.az",
  }
}

```

```

    "user_full_name": "Nicat Əliyev",
    "ip_address": "192.168.1.45",
    "event_type": "MARGIN_CHANGED",
    "module": "COST",
    "action": "BUSINESS",
    "entity_type": "WO_COST_ITEM",
    "entity_id": "uuid-of-cost-item",
    "entity_number": "WO-2026-0042 / Yağ süzgəci",
    "old_value": { "margin_pct": 25.00, "unit_sell_price": 15.63 },
    "new_value": { "margin_pct": 35.00, "unit_sell_price": 16.88 },
    "result": "SUCCESS",
    "created_at": "2026-06-29T11:30:00Z"
  }
}

```

M17.5 Biznes Qaydaları

Kod	Qayda
AUDIT-V01	Audit Log qeydləri heç vaxt silinə bilməz — DELETE endpoint mövcud deyil
AUDIT-V02	Audit Log yalnız AUDIT_READ icazəsi ilə görüntülənə bilər (Admin + Auditor)
AUDIT-V03	old_value / new_value JSONB snapshot saxlanılır — entity silinmiş olsa belə tarixçə qalır
AUDIT-V04	user_email + user_full_name snapshot kimi yazılır — istifadəçi silinsə belə loqda qalır
AUDIT-V05	Yazma asinxron (async) aparılır — ana sorğunun performansına təsir etmir
AUDIT-V06	DENIED result: icazəsiz giriş cəhdi — 403 cavabı verildikdə avtomatik loqlanır
AUDIT-V07	Audit Log saxlama müddəti: minimum 2 il (konfigurasiya edilə bilən)

SOFTWARE REQUIREMENTS SPECIFICATION

CES Service Management System

Sənəd	SRS — Part 6: Infrastructure & Deployment
Versiya	1.0
Tarix	29 İyun 2026
Hazırlayan	Sarkhan Babayev

Docker Compose • Nginx • Flyway Migration • Redis Key Strategiyası • Environment Variables •
Production Checklist

1. Docker Compose Konfigurasiyası

CES Service sistemi Docker Compose ilə tam izolyasiya olunmuş mühitdə işləyir. Development və production üçün ayrı compose faylları var.

1.1 Layihə Qovluq Strukturu

```
ces-service/
├── backend/                # Spring Boot layihəsi
│   ├── src/
│   ├── Dockerfile
│   └── pom.xml
├── frontend/              # Next.js layihəsi
│   ├── app/
│   ├── Dockerfile
│   └── package.json
├── nginx/
│   ├── nginx.conf
│   └── ssl/                # SSL sertifikatları
├── db/
│   └── init/               # İlk başlatma skriptləri
├── docker-compose.yml      # Production
├── docker-compose.dev.yml  # Development override
└── .env.example            # Nümunə env faylı
```

1.2 docker-compose.yml (Production)

```
version: "3.9"

services:

# — PostgreSQL —————
postgres:
  image: postgres:16-alpine
  container_name: ces_service_postgres
  restart: unless-stopped
  environment:
    POSTGRES_DB:      ${POSTGRES_DB}
    POSTGRES_USER:    ${POSTGRES_USER}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./db/init:/docker-entrypoint-initdb.d:ro
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
    interval: 10s
    timeout: 5s
    retries: 5
  networks: [ces_net]

# — Redis —————
redis:
  image: redis:7-alpine
  container_name: ces_service_redis
  restart: unless-stopped
  command: >
    redis-server
    --requirepass ${REDIS_PASSWORD}
    --appendonly yes
    --appendfsync everysec
  volumes:
    - redis_data:/data
  healthcheck:
```

```

    test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5
    networks: [ces_net]

# — Spring Boot Backend —————
backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: ces_service_backend
  restart: unless-stopped
  environment:
    SPRING_DATASOURCE_URL:      jdbc:postgresql://postgres:5432/${POSTGRES_DB}
    SPRING_DATASOURCE_USERNAME: ${POSTGRES_USER}
    SPRING_DATASOURCE_PASSWORD: ${POSTGRES_PASSWORD}
    SPRING_DATA_REDIS_HOST:      redis
    SPRING_DATA_REDIS_PORT:      6379
    SPRING_DATA_REDIS_PASSWORD: ${REDIS_PASSWORD}
    JWT_SECRET:                  ${JWT_SECRET}
    JWT_EXPIRATION_MS:           ${JWT_EXPIRATION_MS}
    JWT_REFRESH_EXPIRATION_MS:   ${JWT_REFRESH_EXPIRATION_MS}
    CES_ERP_BASE_URL:            ${CES_ERP_BASE_URL}
    CES_ERP_API_KEY:             ${CES_ERP_API_KEY}
    STORAGE_BASE_URL:            ${STORAGE_BASE_URL}
    SPRING_PROFILES_ACTIVE:      prod
  depends_on:
    postgres: { condition: service_healthy }
    redis:     { condition: service_healthy }
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8080/actuator/health"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 60s
    networks: [ces_net]

# — Next.js Frontend —————
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
    args:
      NEXT_PUBLIC_API_URL: ${NEXT_PUBLIC_API_URL}
  container_name: ces_service_frontend
  restart: unless-stopped
  environment:
    NEXTAUTH_URL:      ${NEXTAUTH_URL}
    NEXTAUTH_SECRET:   ${NEXTAUTH_SECRET}
    API_BASE_URL:      http://backend:8080
  depends_on:
    backend: { condition: service_healthy }
  networks: [ces_net]

# — Nginx Reverse Proxy —————
nginx:
  image: nginx:1.26-alpine
  container_name: ces_service_nginx
  restart: unless-stopped
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    - ./nginx/ssl:/etc/nginx/ssl:ro
  depends_on:

```



```

    - backend
    - frontend
  networks: [ces_net]

volumes:
  postgres_data:
  redis_data:

networks:
  ces_net:
    driver: bridge

```

1.3 docker-compose.dev.yml (Development Override)

```

version: "3.9"

services:
  postgres:
    ports:
      - "5432:5432"      # Dev-də local bağlantı üçün

  redis:
    ports:
      - "6379:6379"      # Dev-də Redis Insight üçün

  backend:
    build:
      target: dev          # Multi-stage Dockerfile-daki dev stage
    ports:
      - "8080:8080"        # IntelliJ remote debug
      - "5005:5005"        # JVM debug port
    volumes:
      - ./backend/src:/app/src # Hot reload
    environment:
      SPRING_PROFILES_ACTIVE: dev

  frontend:
    ports:
      - "3000:3000"
    volumes:
      - ./frontend:/app
      - /app/node_modules
    command: npm run dev

```

Development başlatma: `docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build`

Production başlatma: `docker compose up -d --build`

1.4 Backend Dockerfile (Multi-stage)

```

# Stage 1: Build
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN ./mvnw package -DskipTests

# Stage 2: Production runtime
FROM eclipse-temurin:21-jre-alpine AS prod
WORKDIR /app
RUN addgroup -S ces && adduser -S ces -G ces

```

```
USER ces
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","-XX:+UseContainerSupport",
            "-XX:MaxRAMPercentage=75.0","app.jar"]
```

1.5 Frontend Dockerfile

```
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
ARG NEXT_PUBLIC_API_URL
ENV NEXT_PUBLIC_API_URL=$NEXT_PUBLIC_API_URL
RUN npm run build

FROM node:20-alpine AS prod
WORKDIR /app
RUN addgroup -S ces && adduser -S ces -G ces
USER ces
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public
EXPOSE 3000
CMD ["node", "server.js"]
```

2. Nginx Konfiqurasiyası

2.1 nginx.conf — Tam Konfiqurasiya

```

worker_processes auto;

events {
    worker_connections 1024;
}

http {
    include        /etc/nginx/mime.types;
    default_type   application/octet-stream;
    sendfile       on;
    gzip           on;
    gzip_types     text/plain application/json application/javascript text/css;

    # Upstream-lər
    upstream backend { server backend:8080; }
    upstream frontend { server frontend:3000; }

    # HTTP → HTTPS yönləndirilməsi
    server {
        listen 80;
        server_name service.ces.az;
        return 301 https://$host$request_uri;
    }

    # Ana HTTPS server bloku
    server {
        listen 443 ssl http2;
        server_name service.ces.az;

        ssl_certificate      /etc/nginx/ssl/fullchain.pem;
        ssl_certificate_key  /etc/nginx/ssl/privkey.pem;
        ssl_protocols        TLSv1.2 TLSv1.3;
        ssl_prefer_server_ciphers on;

        # Security headers
        add_header X-Frame-Options      "SAMEORIGIN" always;
        add_header X-Content-Type-Options "nosniff" always;
        add_header X-XSS-Protection     "1; mode=block" always;
        add_header Referrer-Policy      "strict-origin" always;

        client_max_body_size 50M;      # Fayl yükləmə limiti

        # API → Spring Boot
        location /api/ {
            proxy_pass      http://backend;
            proxy_set_header Host                $host;
            proxy_set_header X-Real-IP           $remote_addr;
            proxy_set_header X-Forwarded-For     $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto   $scheme;
            proxy_set_header Authorization       $http_authorization;
            proxy_set_header X-Branch-Id         $http_x_branch_id;
            proxy_read_timeout 120s;
        }

        # Swagger UI (yalnız dev/staging-də açıq)
        location /swagger-ui/ {
            proxy_pass      http://backend;
            # Production-da: allow 10.0.0.0/8; deny all;
        }
    }
}

```

```
# Actuator (yalnız daxili şəbəkədən)
location /actuator/ {
    allow 172.16.0.0/12;
    deny  all;
    proxy_pass http://backend;
}

# Hər şey → Next.js
location / {
    proxy_pass http://frontend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_http_version 1.1;
}
}
```

3. Flyway Migration Ardıcılığı

Bütün DB dəyişiklikləri Flyway versioned migration skriptləri ilə idarə olunur. Spring Boot başladıqda avtomatik tətbiq olunur.

3.1 Migration Faylları Ardıcılığı

Versiya	Fayl adı	Məzmun
V1	V1__init_schema.sql	Schema yaratma: CREATE SCHEMA ces_service; pgcrypto extension
V2	V2__create_core_tables.sql	branches, users, user_branches cədvəlləri
V3	V3__create_rbac_tables.sql	roles, permissions, role_permissions, user_roles
V4	V4__seed_permissions.sql	51 icazə kodunun INSERT-i (permissions cədvəlinə)
V5	V5__seed_default_roles.sql	6 default rol + default icazə əlaqələri
V6	V6__seed_admin_user.sql	İlk Admin istifadəçi (env variable ilə şifrə)
V7	V7__create_vehicle_tables.sql	vehicles, vehicle_documents
V8	V8__create_customer_tables.sql	customers, contractors
V9	V9__create_service_request_tables.sql	service_requests, service_request_services, service_request_tasks
V10	V10__create_work_order_tables.sql	work_orders, work_order_status_history, work_order_comments, work_order_attachments
V11	V11__create_cost_tables.sql	wo_cost_items, margin_config
V12	V12__create_engine_hour_tables.sql	engine_hour_logs, engine_hour_alerts
V13	V13__add_engine_hours_to_vehicles.sql	vehicles.current_engine_hours + sync trigger
V14	V14__create_inspection_tables.sql	inspection_schedules, inspection_checklists, inspection_checklist_items, inspections, inspection_results
V15	V15__seed_default_checklist.sql	Standart baxış checklist şablonu + 15 maddə
V16	V16__create_warehouse_tables.sql	category_nodes, spare_parts, stock_entries, spare_part_usages, stock_adjustments + triggerlər
V17	V17__create_archive_fields.sql	work_orders-a is_archived, archived_at, reopen_count + trigger
V18	V18__create_document_tables.sql	document_templates, generated_documents
V19	V19__seed_document_templates.sql	5 default sənəd şablonu (boş Thymeleaf şablonu ilə)
V20	V20__create_notification_tables.sql	notifications
V21	V21__create_erp_sync_tables.sql	erp_sync_logs
V22	V22__create_audit_log_table.sql	audit_logs + indexlər

Versiya	Fayl adı	Məzmun
V23	V23__create_sequences.sql	sr_number_seq, wo_number_seq, ins_number_seq + trigger funksiyaları
V24	V24__add_performance_indexes.sql	Bütün əlavə composite indexlər
V25	V25__seed_margin_config.sql	3 default marja konfigurasiyası (LABOR 20%, PART 25%, EXTERNAL 15%)

3.2 Flyway application.yml Konfigurasiyası

```
spring:
  flyway:
    enabled: true
    url: ${SPRING_DATASOURCE_URL}
    user: ${SPRING_DATASOURCE_USERNAME}
    password: ${SPRING_DATASOURCE_PASSWORD}
    schemas: ces_service
    locations: classpath:db/migration
    baseline-on-migrate: false
    validate-on-migrate: true
    out-of-order: false
    table: flyway_schema_history
```

QAYDƏNGİN: Mövcud migration faylını heç vaxt dəyişmə. Hər dəyişiklik = yeni versioned migration. **validate-on-migrate: true** bunu təmin edir.

4. Redis Key Strategiyası

Bütün Redis key-ləri namespace:context:identifier formatında qurulur. TTL hər key tipi üçün ayrıca konfiqurasiya olunur.

4.1 Key Formatları və TTL

Key Format	Məzmun	TTL	Silinmə Halı
auth:blacklist:{jti}	Revoke edilmiş token JTI	Token exp-ə qədər	Avtomatik (TTL)
auth:refresh:{userId}:{jti}	Refresh token store	7 gün	Logout / şifrə dəyişikliyi
auth:ratelimit:{ip}	Brute-force cəhd sayı	15 dəq	Avtomatik (TTL)
auth:locked:{email}	Kilidlənmiş hesab	15 dəq	Avtomatik (TTL)
perm:{userId}:{branchId}	İstifadəçi icazə siyahısı (JSON array)	15 dəq	Rol dəyişikliyinə manual silmə
session:{userId}	Aktiv session metadata	1 saat	Logout zamanı
cache:dashboard:{branchId}	Dashboard KPI xülasəsi	60 saniyə	Avtomatik (TTL)
cache:customers:{branchId}:{page}	ERP müştəri siyahısı cache	5 dəq	Sinxronizasiyadan sonra
cache:eh:latest:{vehicleId}	Son motosaat dəyəri	10 dəq	Yeni qeyd yazıldıqda
cache:lowstock:{branchId}	Aşağı stok siyahısı	5 dəq	Stok dəyişikliyindən sonra
erp:retry:{syncLogId}	ERP retry queue item	24 saat	Uğurlu sync-dən sonra
notif:unread:{userId}	Oxunmamış bildiriş sayı	5 dəq	Bildiriş oxunanda

4.2 Redis Java Konfiqurasiyası

```
// RedisConfig.java
@Configuration
public class RedisConfig {

    @Bean
    public RedisTemplate<String, Object> redisTemplate(
        RedisConnectionFactory factory) {
        RedisTemplate<String, Object> t = new RedisTemplate<>();
        t.setConnectionFactory(factory);
        t.setKeySerializer(new StringRedisSerializer());
        t.setValueSerializer(new GenericJackson2JsonRedisSerializer());
        t.setHashKeySerializer(new StringRedisSerializer());
        t.setHashValueSerializer(new GenericJackson2JsonRedisSerializer());
        return t;
    }

    @Bean
    public CacheManager cacheManager(RedisConnectionFactory factory) {
        Map<String, RedisCacheConfiguration> configs = Map.of(
            "dashboard", ttl(60),
            "customers", ttl(300),
            "eh_latest", ttl(600),
            "lowstock", ttl(300),
            "permissions",ttl(900)
        );
        return RedisCacheManager.builder(factory)
```

```
        .withInitialCacheConfigurations(configs).build();  
    }  
  
    private RedisCacheConfiguration ttl(long seconds) {  
        return RedisCacheConfiguration.defaultCacheConfig()  
            .entryTtl(Duration.ofSeconds(seconds));  
    }  
}
```


5. Environment Variables

.env faylı versiyalama sisteminə (Git) əlavə edilməməlidir. .env.example faylı nümunə kimi repository-də saxlanılır.

Qaydaqoyan: .env → .gitignore-a əlavə et. CI/CD mühitdə GitHub Secrets / GitLab CI Variables istifadə et.

5.1 .env.example — Tam Siyahı

```
# =====
# CES Service Management System - Environment Variables
# Kopyala: cp .env.example .env
# =====

# — PostgreSQL —————
POSTGRES_DB=ces_service_db
POSTGRES_USER=ces_service_user
POSTGRES_PASSWORD=CHANGE_ME_STRONG_PASSWORD

# — Redis —————
REDIS_PASSWORD=CHANGE_ME_REDIS_PASSWORD

# — JWT —————
JWT_SECRET=CHANGE_ME_256BIT_BASE64_SECRET
JWT_EXPIRATION_MS=3600000 # 1 saat (millisaniyə)
JWT_REFRESH_EXPIRATION_MS=604800000 # 7 gün (millisaniyə)

# — CES ERP İntegrasiya —————
CES_ERP_BASE_URL=https://erp.ces.az
CES_ERP_API_KEY=CHANGE_ME_ERP_API_KEY

# — Fayl Saxlama (Storage) —————
STORAGE_BASE_URL=https://storage.ces.az
STORAGE_ACCESS_KEY=CHANGE_ME
STORAGE_SECRET_KEY=CHANGE_ME
STORAGE_BUCKET_NAME=ces-service-files

# — Next.js / NextAuth —————
NEXT_PUBLIC_API_URL=https://service.ces.az
NEXTAUTH_URL=https://service.ces.az
NEXTAUTH_SECRET=CHANGE_ME_32CHAR_SECRET

# — Admin İlk İstifadəçi (V6 migration üçün) —————
ADMIN_INIT_EMAIL=admin@ces.az
ADMIN_INIT_PASSWORD=CHANGE_ME_ADMIN_PASSWORD
ADMIN_INIT_FULL_NAME=System Administrator

# — Spring Profil —————
SPRING_PROFILES_ACTIVE=prod

# — Log Səviyyəsi —————
LOG_LEVEL=INFO # dev-də DEBUG
```

5.2 Mühüm Dəyərlər üçün Generasiya Əmrləri

JWT_SECRET — 256-bit base64 generasiya:

```
openssl rand -base64 32
```

NEXTAUTH_SECRET — 32 simvol:

```
| openssl rand -base64 24
```

POSTGRES_PASSWORD — güclü şifrə:

```
| openssl rand -base64 20 | tr -dc 'A-Za-z0-9!@#' | head -c 24
```

6. Production Checklist

Sistemin production-a çıxarılmazdan əvvəl aşağıdakı bütün maddələr yoxlanılmalı və təsdiqlənməlidir.

6.1 İnfrastruktur & Təhlükəsizlik

	Yoxlama	Status
1	.env faylı Git-dən kənardadır (.gitignore-da)	☐
2	Bütün CHANGE_ME dəyərləri əvəz edilib	☐
3	JWT_SECRET ən az 256-bit random dəyərdir	☐
4	HTTPS aktiv — SSL sertifikatı qurulub	☐
5	HTTP → HTTPS yönləndirilməsi işləyir	☐
6	Security header-lər (X-Frame-Options, HSTS...) Nginx-də aktivdir	☐
7	Actuator endpoint-i yalnız daxili şəbəkədən əlçatandır	☐
8	Swagger UI production-da deaktiv edilib (spring.springdoc.enabled=false)	☐
9	PostgreSQL port-u (5432) xarici şəbəkəyə açıq deyil	☐
10	Redis port-u (6379) xarici şəbəkəyə açıq deyil	☐
11	Docker container-lar root olmayan istifadəçi ilə işləyir	☐
12	POSTGRES_PASSWORD bcrypt ilə deyil — DB şifrəsi güclü random-dur	☐

6.2 Verilənlər Bazı & Migration

	Yoxlama	Status
1	Flyway migration-lar uğurla tamamlandı (flyway_schema_history yoxlanılıb)	☐
2	25 migration faylı hamısı SUCCESS statusundadır	☐
3	51 icazə kodu permissions cədvəlindədir	☐
4	6 default rol yaradılıb və icazələr düzgün təyin edilib	☐
5	İlk Admin istifadəçi yaradılıb və giriş test edilib	☐
6	3 default marja konfigurasiyası (LABOR/PART/EXTERNAL) yazılıb	☐
7	DB backup strategiyası qurulub (gündəlik cron)	☐
8	Test DB-sinin production-a qarışmadığı yoxlanılıb	☐

6.3 API & Backend

	Yoxlama	Status
1	GET /actuator/health → {"status":"UP"} qaytarır	☐

	Yoxlama	Status
2	POST /api/v1/auth/login uğurla işləyir	☐
3	JWT token imzalanır və doğrulana bilir	☐
4	X-Branch-Id header olmadan API 400/403 qaytarır	☐
5	İcazəsiz endpoint-ə giriş 403 qaytarır (PERMISSION_DENIED)	☐
6	Rate limiting işləyir (5 uğursuz cəhd → 429)	☐
7	CORS konfigurasiyası yalnız service.ces.az-a icazə verir	☐
8	PDF generasiya testi uğurlu (service act yaradılıb)	☐
9	CES ERP bağlantısı test edilib (müşəri sync işləyir)	☐
10	Audit Log yazılır (login hadisəsi yoxlanılıb)	☐

6.4 Frontend & İstifadəçi Axını

	Yoxlama	Status
1	Login səhifəsi açılır və giriş işləyir	☐
2	Sistem seçimi (İcarə / Texniki Servis) ekranı görünür	☐
3	Branch seçici (çoşşöbəli) düzgün işləyir	☐
4	Rol əsaslı menyu görünürüyü test edilib (mexanik vs manager)	☐
5	Dashboard KPI-lar düzgün yüklənir	☐
6	Work Order yaratma axını başdan sona test edilib	☐
7	Anbar stok çıxışı → WO-ya bağlı xərc hesablaması işləyir	☐
8	PDF sənəd yükləmə testi uğurludur	☐
9	Bildiriş siyahısı açılır	☐
10	Mobil responsivlik (tablet + telefon) yoxlanılıb	☐

6.5 Monitoring & Log

	Yoxlama	Status
1	Spring Boot application.log faylı yazılır	☐
2	Log səviyyəsi production-da INFO-dur	☐
3	Docker container log-ları əlçatandır (docker logs)	☐
4	Redis AOF persistence aktiv (appendonly yes)	☐
5	Docker volume-lar host-da düzgün mount edilib	☐
6	restart: unless-stopped bütün container-larda var	☐
7	Disk sahəsi yetərlidir (log + DB + backup üçün)	☐

7. Tam Cədvəl Siyahısı (ces_service schema)

Aşağıda sistemin bütün 29 cədvəli, hansı modul tərəfindən istifadə edildiği və əsas münasibətlər göstərilmişdir.

#	Cədvəl adı	Modul	Açıqlama & Əsas Münasibətlər
1	branches	Foundation	Şöbələr — bütün cədvəllər branch_id ilə istinad edir
2	users	M15	İstifadəçilər — failed_attempts, locked_until
3	user_branches	M15	İstifadəçi ↔ Branch (çoxlu şöbə dəstəyi)
4	roles	M16	Dinamik rol kataloqu — is_system flag
5	permissions	M16	51 icazə kodu — perm_type: CRUD/BUSINESS
6	role_permissions	M16	Rol ↔ İcazə M2M əlaqəsi
7	user_roles	M16	User ↔ Role ↔ Branch üçlü PK
8	vehicles	M03	Texnikalar — garage_type: COMPANY/INVESTOR/CUSTOMER
9	vehicle_documents	M03	Texnika sənədləri — file_url, expires_at
10	customers	M04	Müştərilər — erp_customer_id sinxronizasiya üçün
11	contractors	M05	Podratçılar — vasitəçilik sorğuları üçün
12	service_requests	M05	3 növ sorğu — intermediary_profit GENERATED
13	service_request_services	M05	Pullu xidmət siyahısı — FIXED/HOURLY
14	service_request_tasks	M05	Şirkət sorğusu dinamik tapşırıqları
15	work_orders	M06	Work Order — status axını, is_archived, reopen_count
16	work_order_status_history	M06	Hər status keçidinin tarixçəsi
17	work_order_comments	M06	WO şərhləri
18	work_order_attachments	M06	WO fayl əlavələri
19	wo_cost_items	M07	Xərc maddələri — LABOR/PART/EXTERNAL/OTHER
20	margin_config	M07	Branch üzrə default marja konfigurasiyası
21	engine_hour_logs	M08	Motosaat tarixçəsi — delta GENERATED, is_rollover
22	engine_hour_alerts	M08	Motosaat xəbərdarlıq konfigurasiyası
23	inspection_schedules	M18	Baxış planı — HOURS/DAYS/BOTH trigger
24	inspection_checklists	M18	Baxış checklist şablonları
25	inspection_checklist_items	M18	Şablon maddələri — JSONB options
26	inspections	M18	Baxış qeydləri — generated_wo_id bağlantısı
27	inspection_results	M18	Hər maddənin nəticəsi — severity: OK/WARNING/CRITICAL
28	category_nodes	M19	Anbar qovluq ağacı — self-referencing, path, depth
29	spare_parts	M19	Ehtiyat hissəsi kataloqu — low_stock partial index
30	stock_entries	M19	Stok giriş qeydləri — unit_cost update trigger

#	Cədvəl adı	Modul	Açıqlama & Əsas Münasibətlər
31	spare_part_usages	M19	WO-bağlı stok çıxışı — tarixi unit_cost qorunur
32	stock_adjustments	M19	Stok düzəltmə — Audit Log-a yazılır
33	document_templates	M10	PDF şablonları — Thymeleaf HTML
34	generated_documents	M10	Generasiya olunmuş sənədlər — doc_number unikal
35	notifications	M12	In-app bildirişlər — ref_type/ref_id pattern
36	erp_sync_logs	M13	ERP sinxronizasiya tarixçəsi — retry count
37	audit_logs	M17	Immutable audit qeydləri — old/new JSONB snapshot

8. SRS Xülasəsi

8.1 Part-lar Üzrə İcmal

Part	Ad	Əhatə
1	Foundation	System Overview, Tech Stack, DB Konvensiyaları, Auth Arxitekturası, Multi-Branch Model, API Konvensiyaları, Layihə Strukturu
2	Core Modules	M03 Qaraj, M04 Müştəri, M05 Servis Sorğuları (3 növ), M06 Work Order, M07 Xərc & Marja
3	Service Operations	M08 Motosaat, M18 Texniki Baxış, M19 Anbar & Ehtiyat Hissələri
4	Completion & Output	M09 Arxiv, M10 Sənəd Generasiyası, M11 Hesabatlar, M12 Bildiriş, M13 ERP İntegrasiyası, M14 Mobil API
5	Administration	M15 İstifadəçi İdarəetməsi, M16 RBAC (51 icazə + tam matris), M17 Audit Log
6	Infrastructure	Docker Compose, Nginx, Flyway (25 migration), Redis Key Strategiyası, Environment Variables, Production Checklist

8.2 Rəqəmsal İcmal

Göstərici	Dəyər
Əhatə olunan modul sayı	19 modul (M01–M19)
DB cədvəlləri	37 cədvəl (ces_service schema)
Flyway migration fayl	25 versioned migration
İcazə kodu (permissions)	51 unikal permission kodu
Default rol	6 (Admin, Service Manager, Mechanic, Accountant, Warehouse, Director)
API endpoint (təxmini)	110+ endpoint (bütün modullar üzrə)
DB trigger	8 PostgreSQL trigger (stok, motosaat, arxiv, nömrələmə...)
Biznes validasiya qaydası	70+ validasiya qaydası (modul üzrə)
Xəta kodu	16+ standart xəta kodu
Audit hadisə tipi	23 audit event_type
Redis key tipi	13 unikal key format
Docker servis	4 (postgres, redis, backend, frontend, nginx)

8.3 Növbəti Addımlar

- SRS Part 1–6 sənədləri birləşdirilərək vahid SRS v1.0 sənədi hazırlanır
- Development komandası SRS-i nəzərdən keçirir — suallar və dəqiqləşmələr toplanır
- Flyway V1 migration ilə DB sxemi yaradılır
- Spring Boot layihəsi hazırlanır — BaseEntity, auth filter zənciri, RBAC
- M03 (Qaraj) və M05 (Servis Sorğuları) ilk inkişaf sprint-ləri — MVP
- Frontend Next.js layihəsi qurulur — auth axını, branch seçici, dashboard

- Faza 1: M01–M09 + M18 + M19 (core funksionallıq)
- Faza 2: M10–M14 (sənəd, hesabat, bildiriş, ERP, mobil)
- Faza 3: SMS/WhatsApp, elektron imza, QR, mobil tam versiya

SRS sənədi statik deyil — inkişaf prosesində yaranan suallar əsasında yenilənəcək. Hər yenilik revision history-yə əlavə olunur.